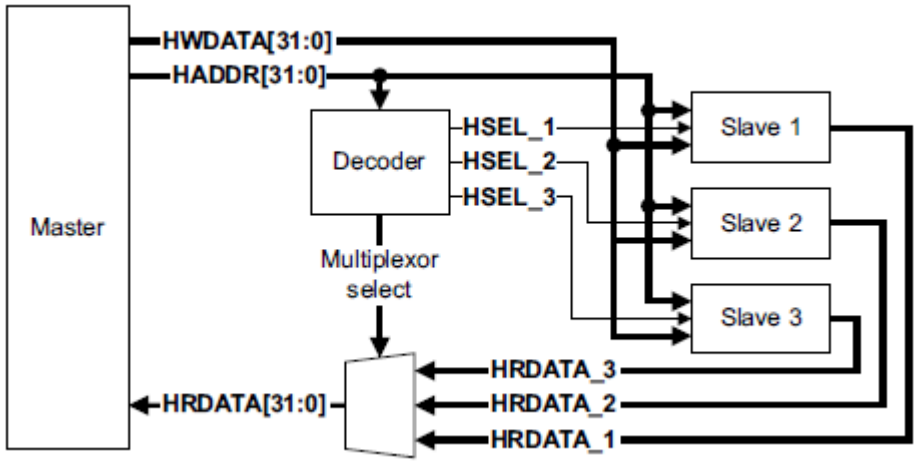
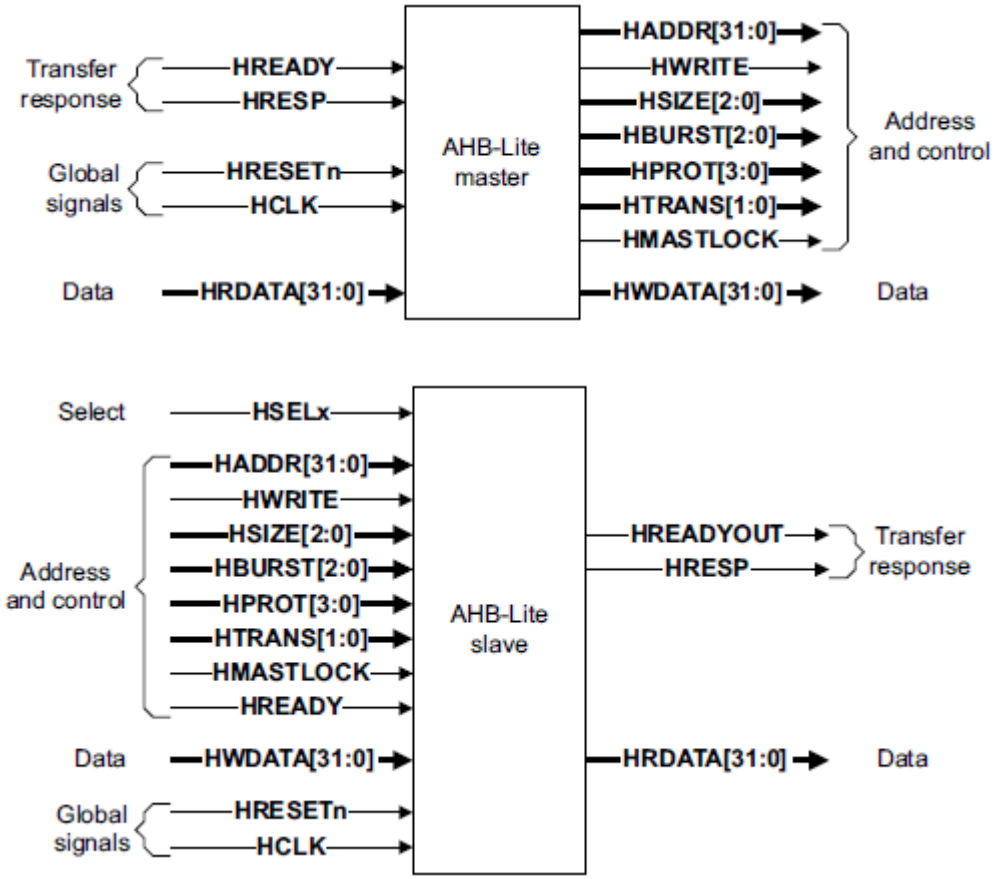
AMBA 3 AHB-Lite Protocol

AHB-Lite block diagram

- AMBA AHB-Lite It is a bus interface that supports a ***single bus master*** and provides high-bandwidth operation.

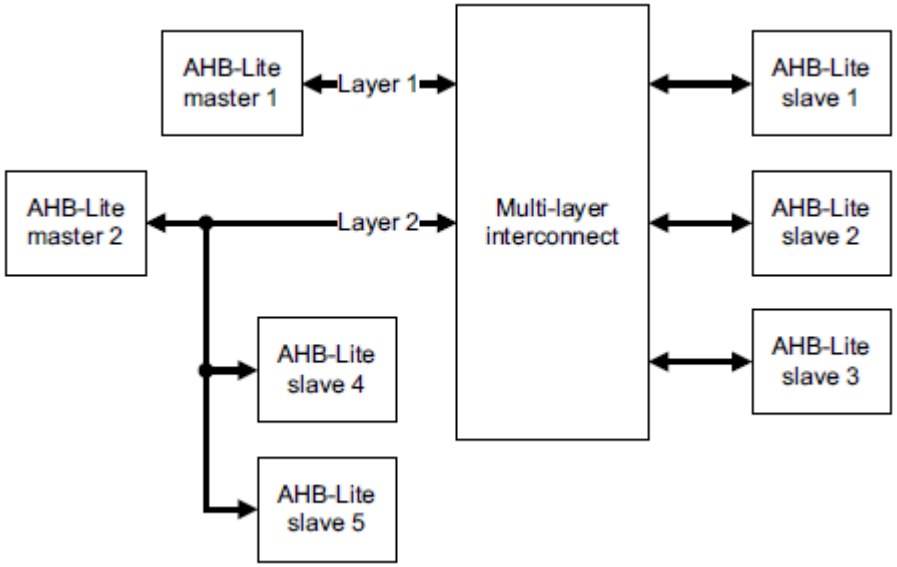


Master & slave interface



Multi-layer AHB-lite

- Parallel bus implementation
  - May require some arbitration on the slave side if multiple masters access the same slaves at the same time.



Multi-layer AHB

Signal conventions

- Multi-layer AHB is an interconnection scheme, based on the AHB protocol, that enables parallel access paths between multiple masters and slaves in a system.
- Each AHB layer can be very simple because it only has one master
  - no arbitration or master-to-slave muxing is required.
  - These layers can use the AHB-Lite protocol, meaning that they do not have to support request and grant, or retry and split transactions.
  - The arbitration effectively becomes **point arbitration** at each peripheral and is only necessary when more than one master wants to access the same slave simultaneously.

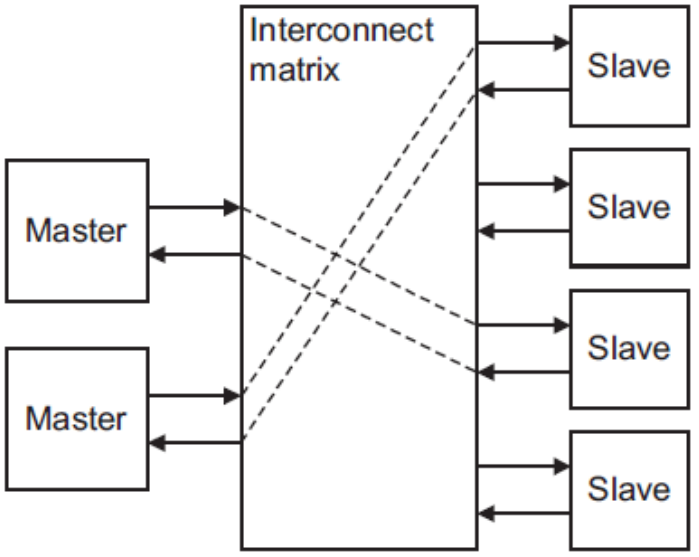
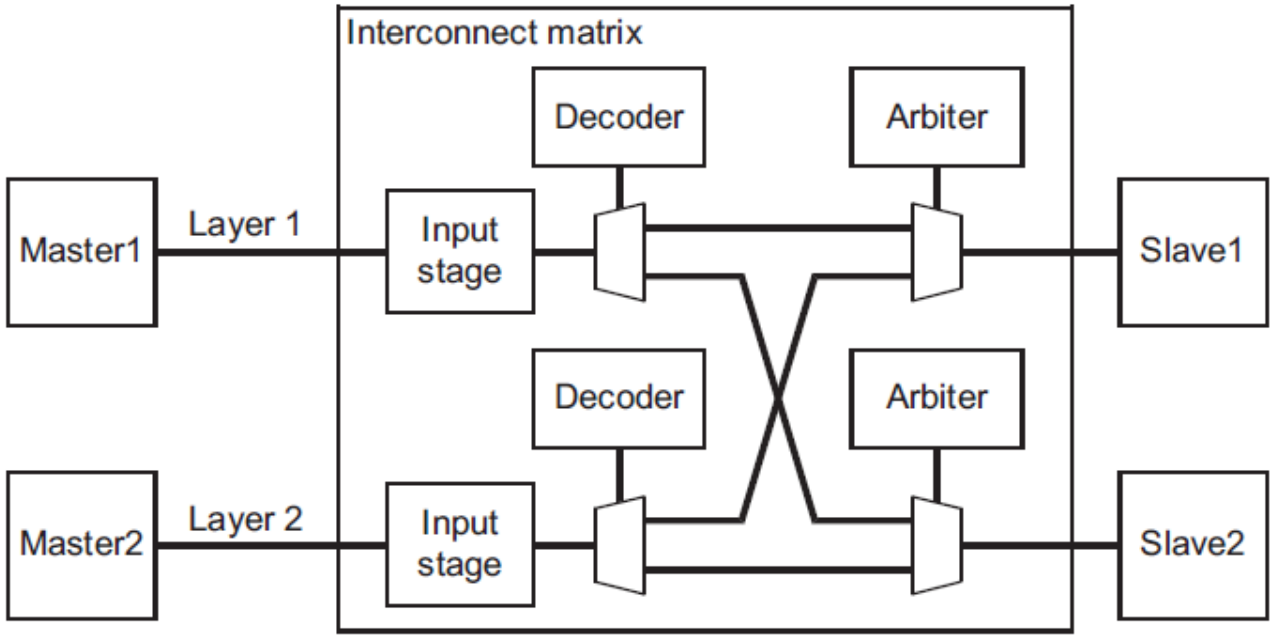


Figure 1 Basic multi-layer concept

Implementation

- Within the interconnect matrix:
  - Every layer has a Decode stage that determines which slave is required for a transfer.
  - A mux routes the transfer from the appropriate layer to the required slave.



Bus configuration

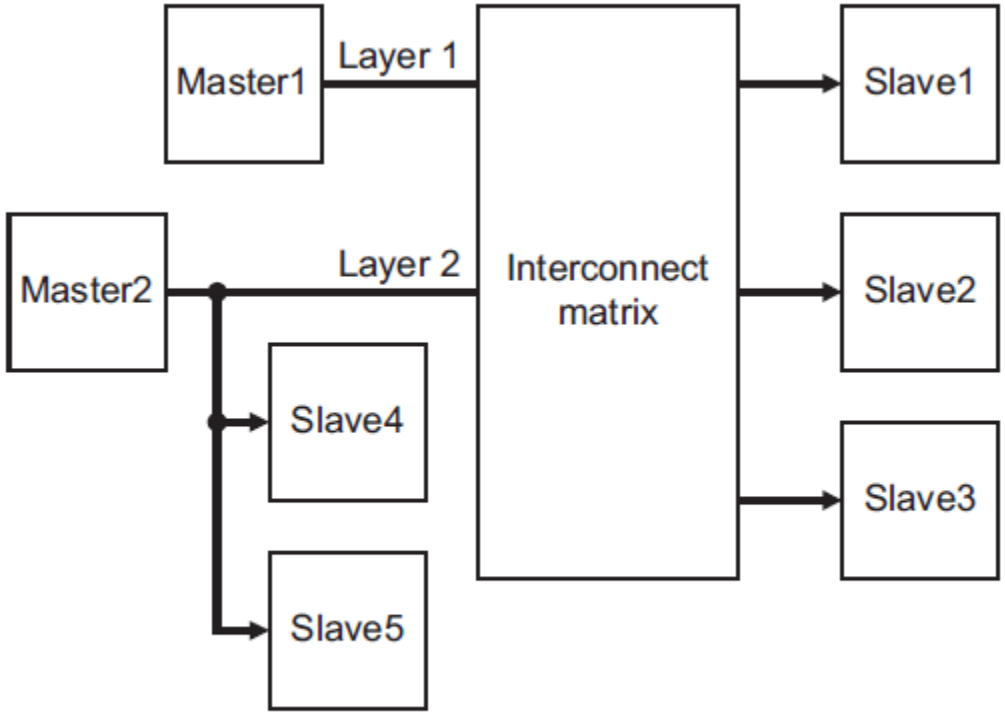


Figure 3 Local slaves

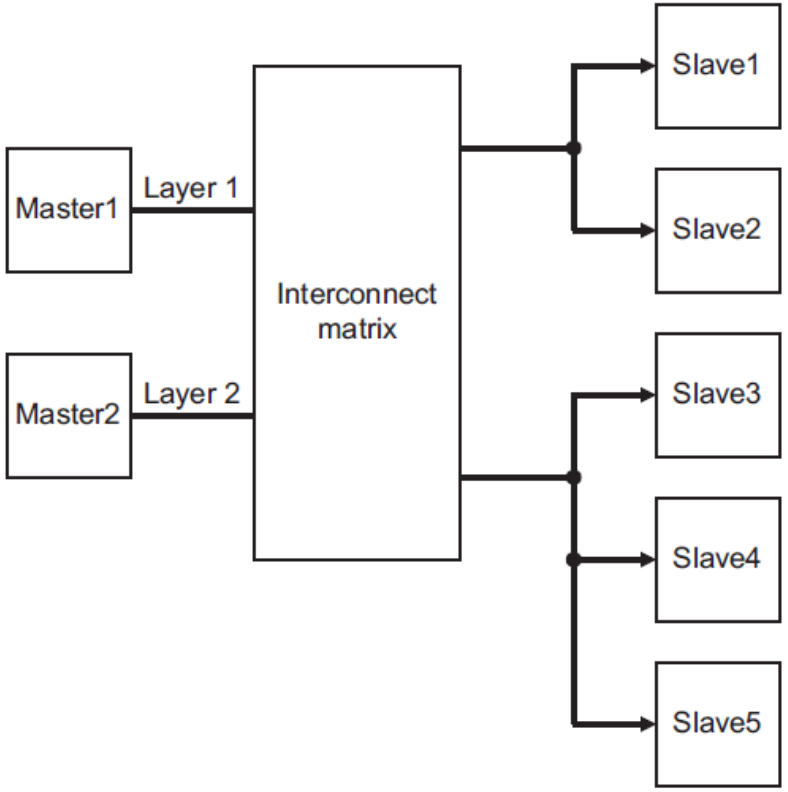


Figure 4 Multiple slaves on one slave port

Bus configuration

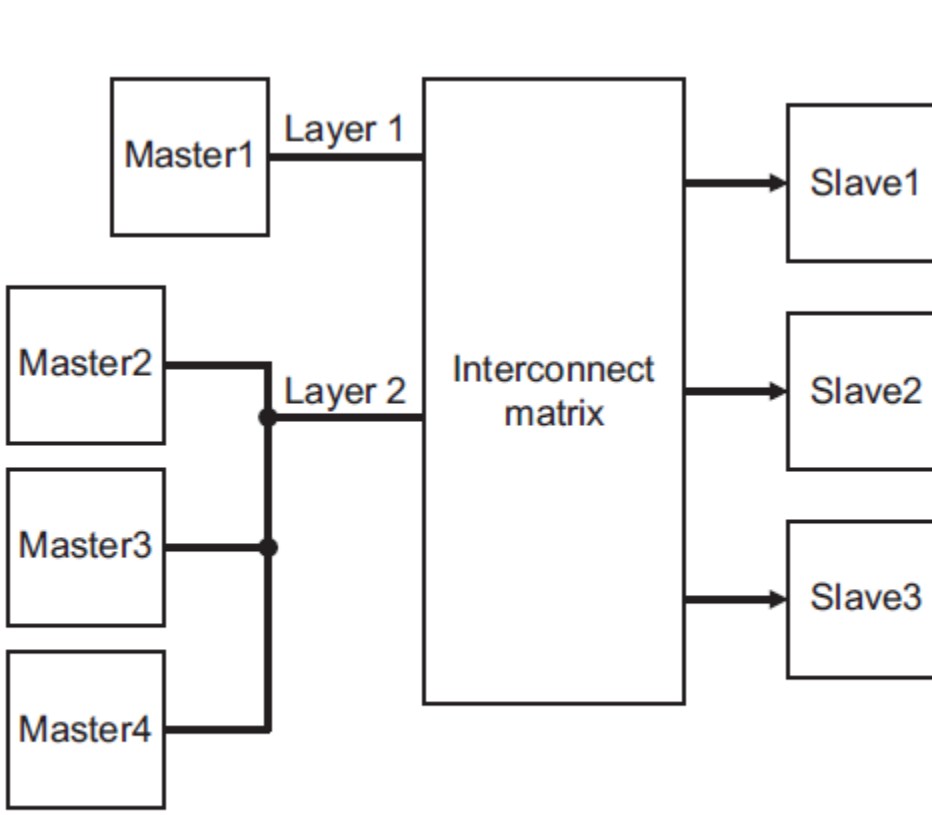


Figure 5 Multiple masters on one layer

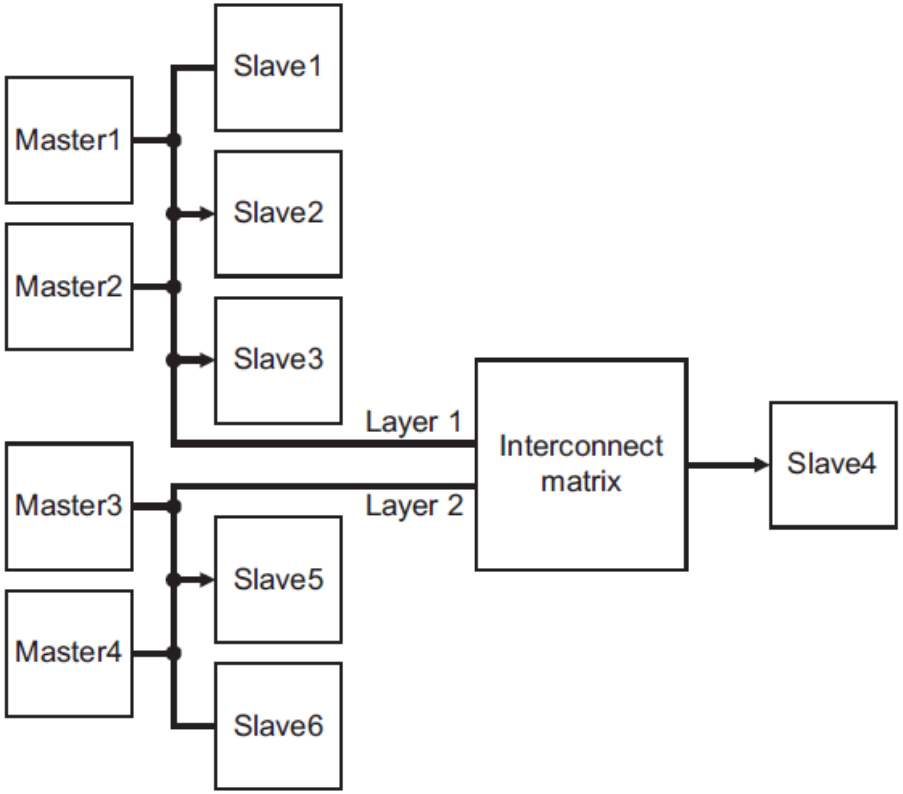
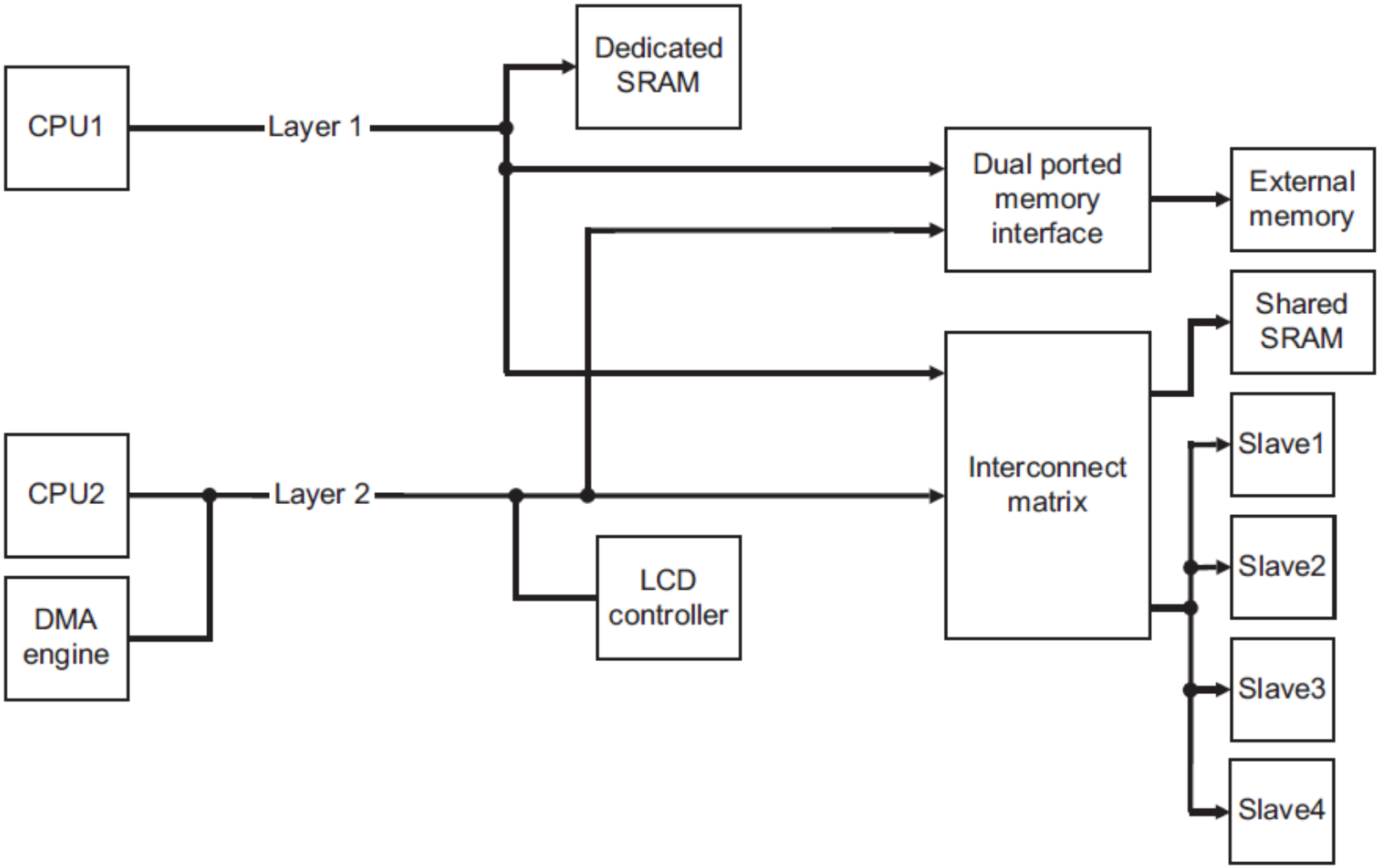


Figure 6 Separate AHB subsystems

Example implementation



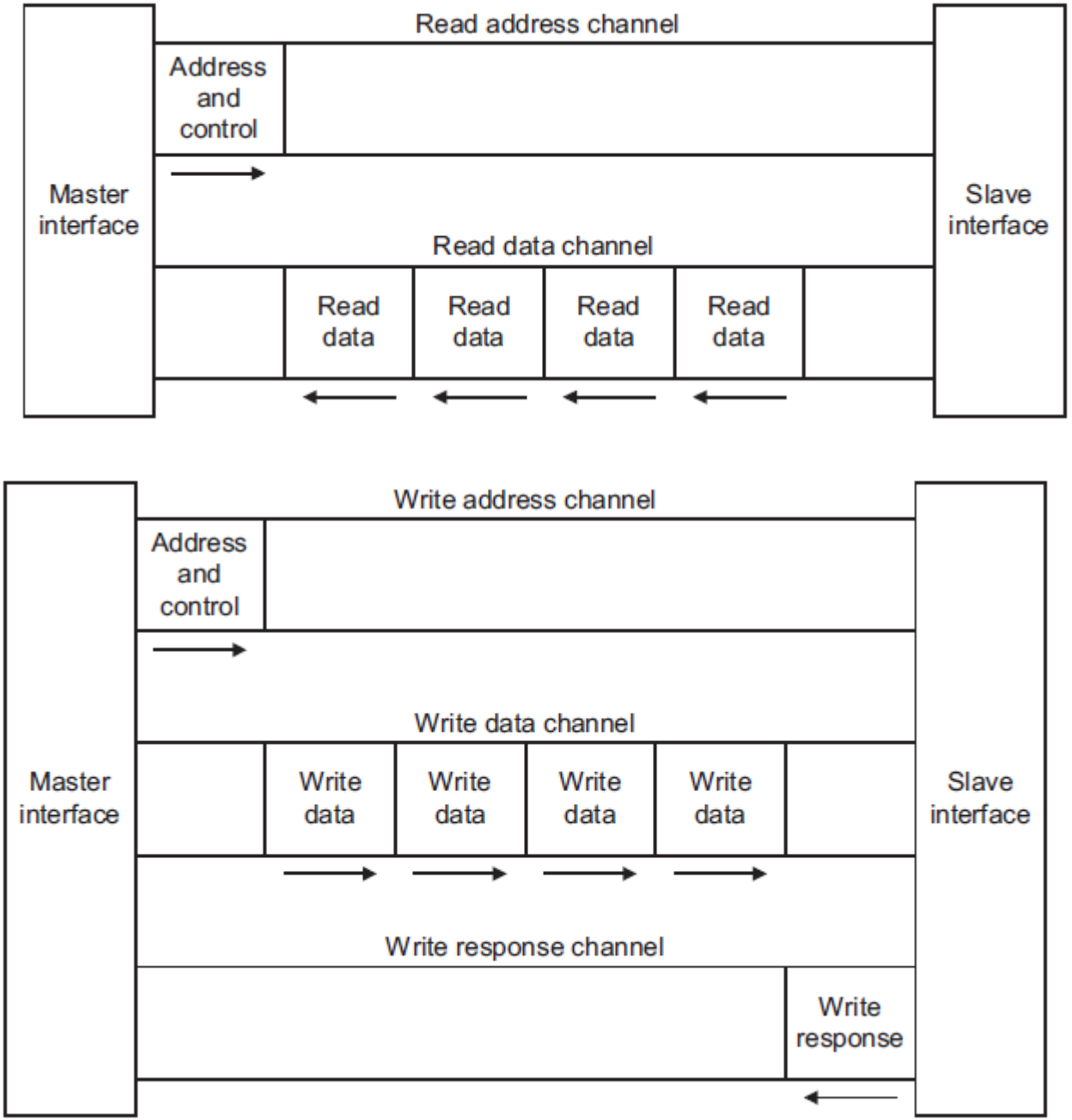
AXI Bus

Objectives of AXI interface

- Suitable for high-bandwidth and low-latency design
- Enable high-frequency operation without using complex bridges
- meet requirements of wide range of components
- Suitable for memory controllers with high initial access latency
- More flexibility in the implementation of interconnect architectures
- Backward-compatible with existing AHB and APB interface

AXI features

- AXI – Advanced eXtensible Interface
- Burst based
- The AXI protocol enables:
  - address issued ahead of the actual data transfer
  - multiple outstanding transactions
  - out-of-order completion of transactions.
- Five channels
  - Read address channel
  - Read data channel
  - Write address channel
  - Write data channel
  - Write response channel


Read address channel
Address and control
Master interface
Slave interface
Read data channel
Read data
Read data
Read data
Read data
Write address channel
Address and control
Write data channel
Master interface
Write data
Write data
Write data
Write data
Slave interface
Write response channel
Write response

AXI features

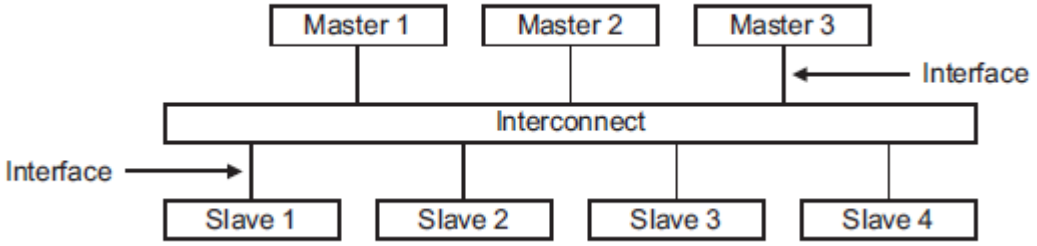
- Two-way VALID (issued from source) and READY (issued from destination) handshake mechanism
- LAST issued to indicated the final data.
- The AXI protocol supports
  - variable-length bursts (from 1 to 16 data transfers)
  - a transfer size of 8-1024 bits
  - wrapping, incrementing, and non-incrementing bursts
  - atomic operations, using exclusive or locked accesses
  - system-level caching and buffering control

Data channel

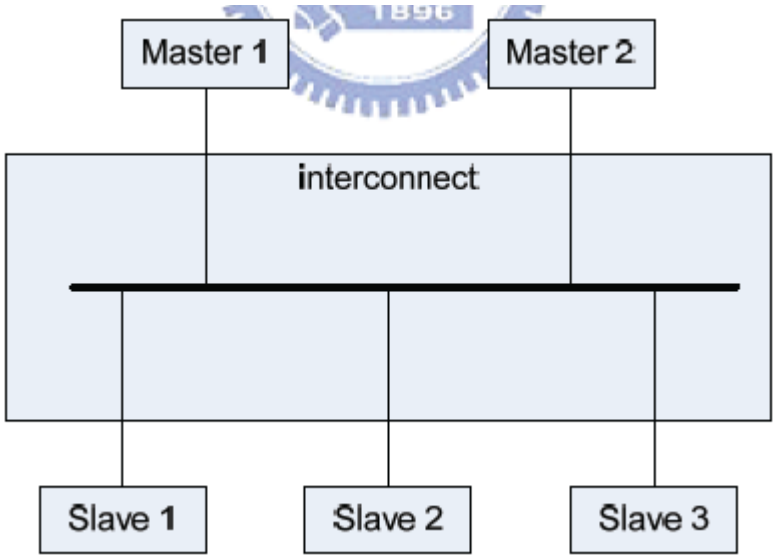
- Read data channel
  - data bus can be 8, 16, 32, 64, 128, 256, 512, or 1024 bits wide
  - a read response (LAST) indicating the completion status of the read transaction
- Write data channel
  - data bus can be 8, 16, 32, 64, 128, 256, 512, or 1024 bits wide
  - one byte lane strobe for every eight data bits, indicating which bytes of the data bus are valid
    - Write data channel information is always treated as buffered, so that the master can perform write transactions without slave acknowledgement of previous write transactions.

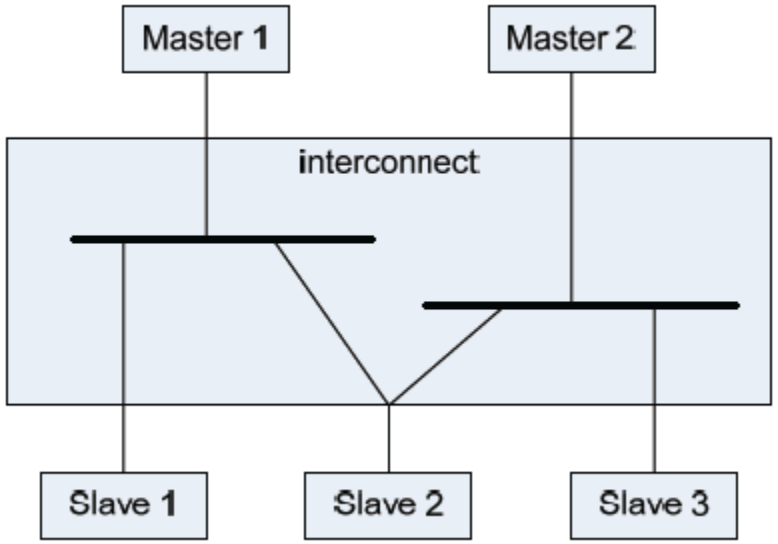
Interface & Interconnect

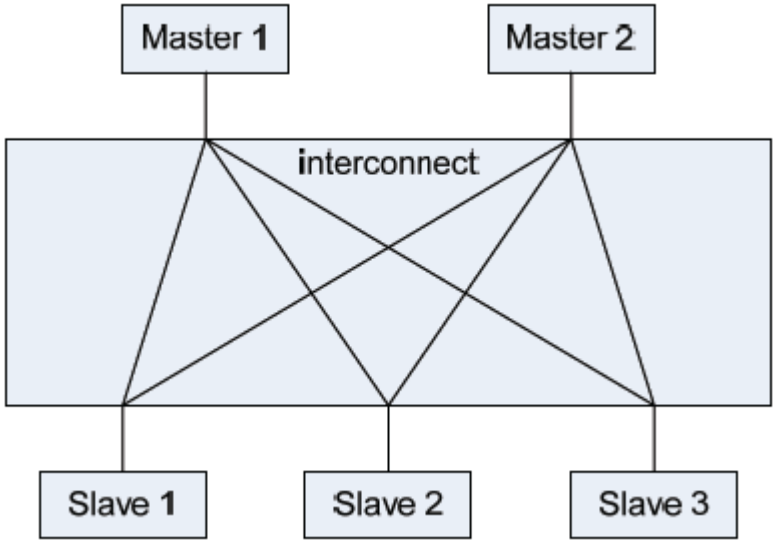
- The AXI protocol provides a single interface definition for describing interfaces:
  - between a master and the interconnect
  - between a slave and the interconnect
  - between a master and a slave.
- Most systems use one of three interconnect approaches:
  - shared address and data buses
  - shared address buses and multiple data buses
  - multilayer, with multiple address and data buses.



Physical bus architecture







Register Slices

- Each AXI channel transfers information in only one direction, and there is no requirement for a fixed relationship between the various channels.
- This feature enables the insertion of a register slice in any channel, at the cost of an additional cycle of latency.
  - This makes possible a trade-off between cycles of latency and maximum frequency of operation.
  - It is also possible to use register slices at almost any point within a given interconnect.
  - It can be advantageous to use a direct, fast connection between a processor and high-performance memory, but to use simple register slices to isolate a longer path to less performance-critical peripherals.

Signal conventions

- Prefix A Denotes AXI global, write address, and read address channel signals.
  - ACLK, AWADDR, ARADDR
- Prefix B Denotes AXI write response channel signals.
- Prefix C Denotes AXI low-power interface signals.
- Prefix R Denotes AXI read data channel signals.
- Prefix W Denotes AXI write data channel signals.
- Suffix n Denotes that the AXI ARESETn global reset

Read burst example

- A pair of Valid-READY handshake messages are used to denote a transaction.
- The slave keeps the VALID signal LOW until the read data is available.

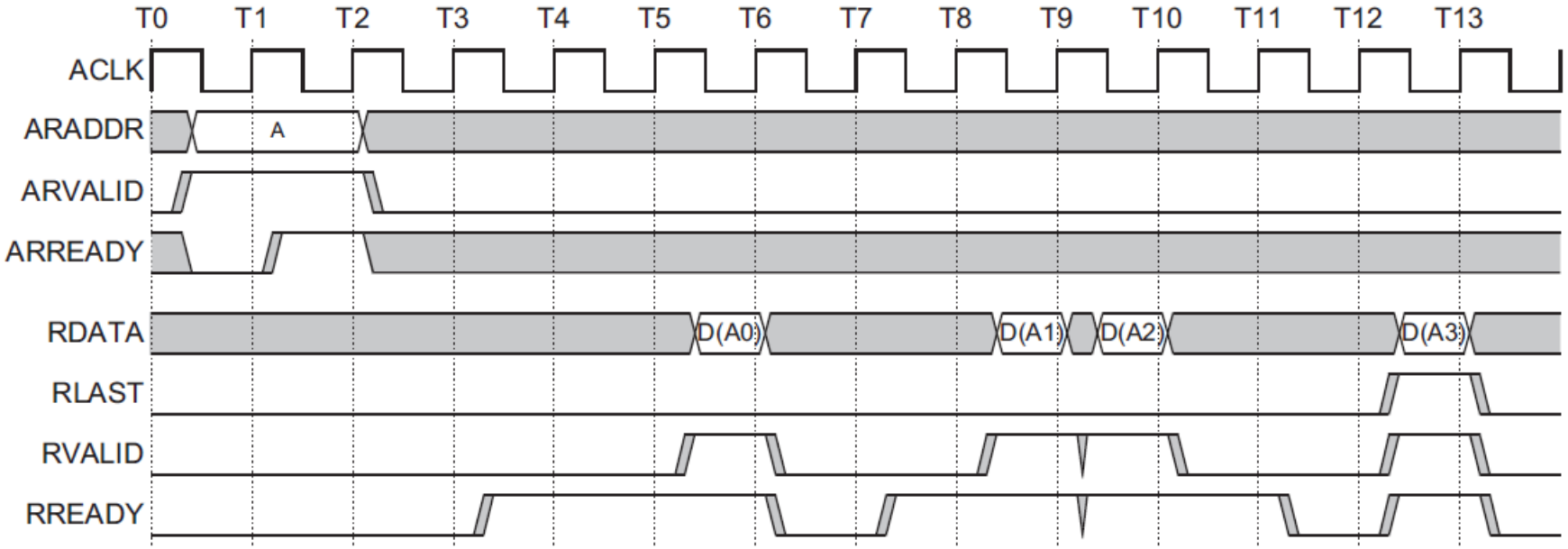
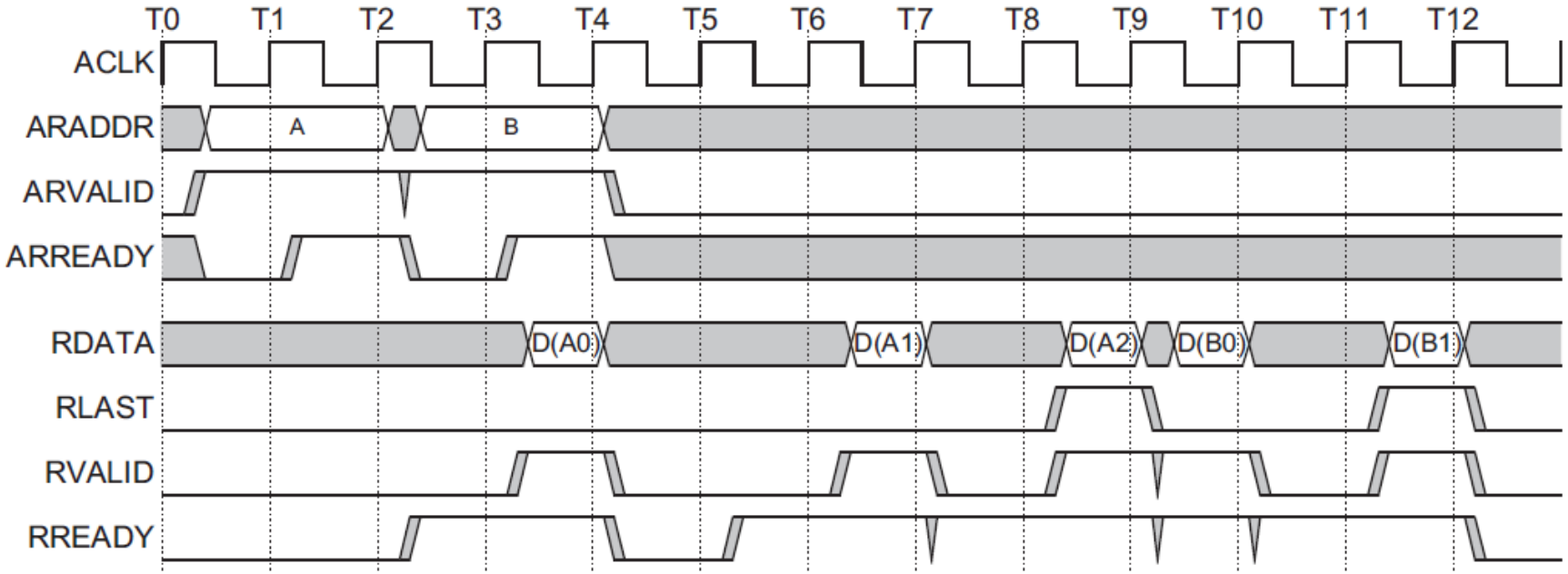


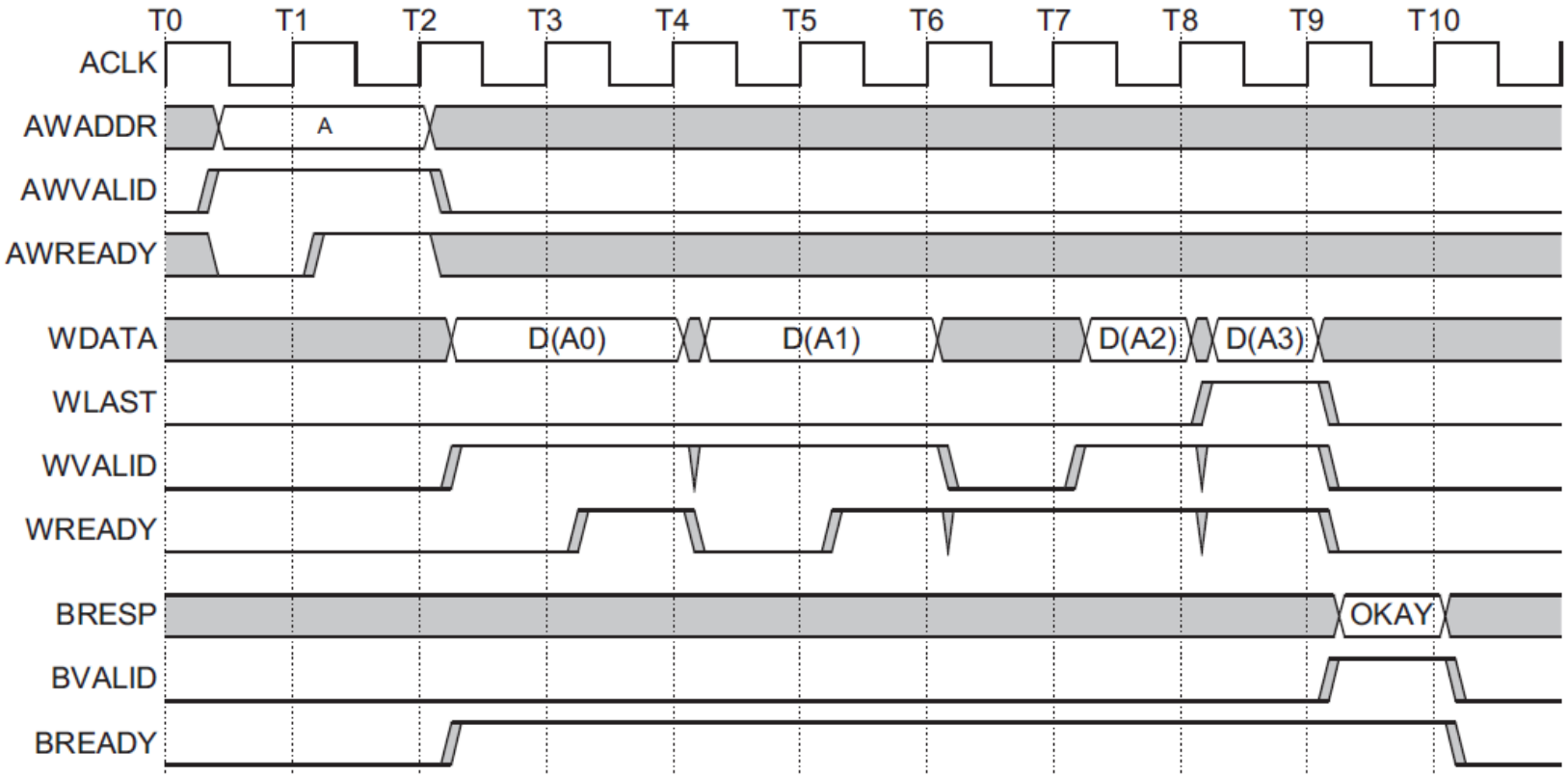
Figure 1-4 Read burst

Overlapping read bursts

- RLAST is used to denote the end of the burst transfer.
- The slave keeps the VALID signal LOW until the read data is available.

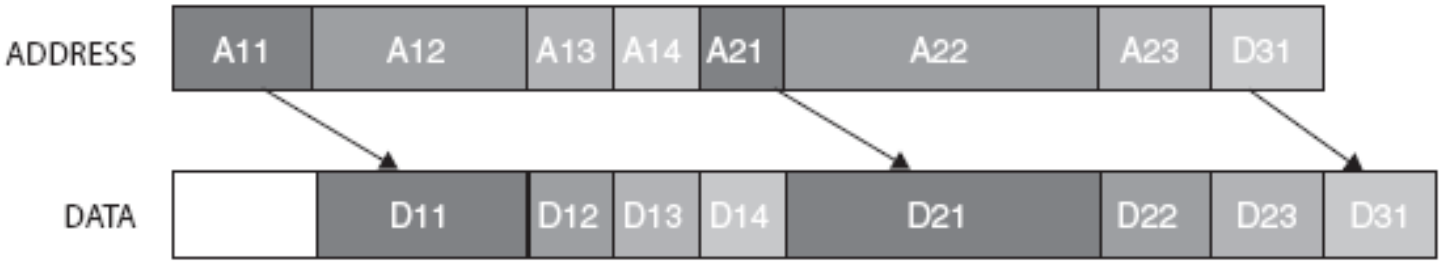


Write burst

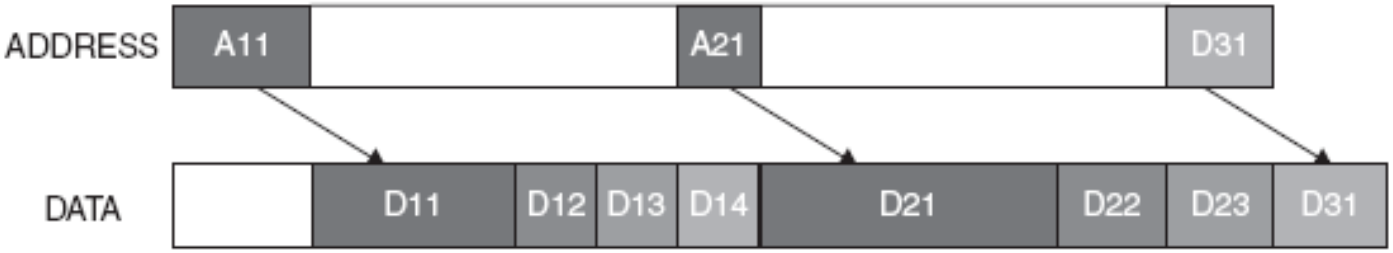

T0
T1
T2
T3
T4
T5
T6
T7
T8
T9
T10
ACLK
AWADDR
A
AWVALID
AWREADY
WDATA
D(A0)
D(A1)
D(A2)
D(A3)
WLAST
WVALID
WREADY
BRESP
OKAY
BVALID
BREADY

AHB vs. AXI Burst

- AHB Burst
  - Address and Data are locked together (single pipeline stage)
  - HREADY controls intervals of address and data



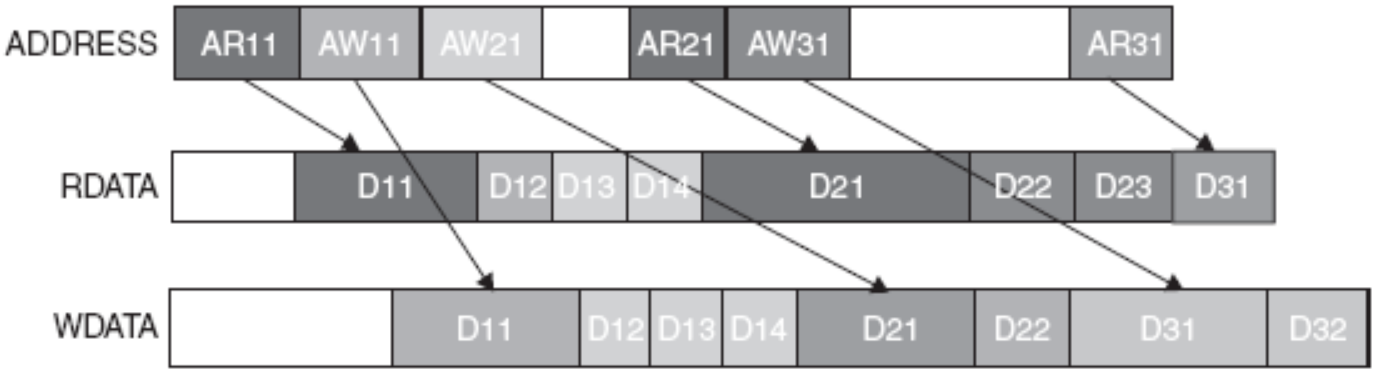
- AXI Burst
  - One Address for entire burst



© 2008 Sudeep Pasricha & Nikil Dutt

AHB vs. AXI Burst

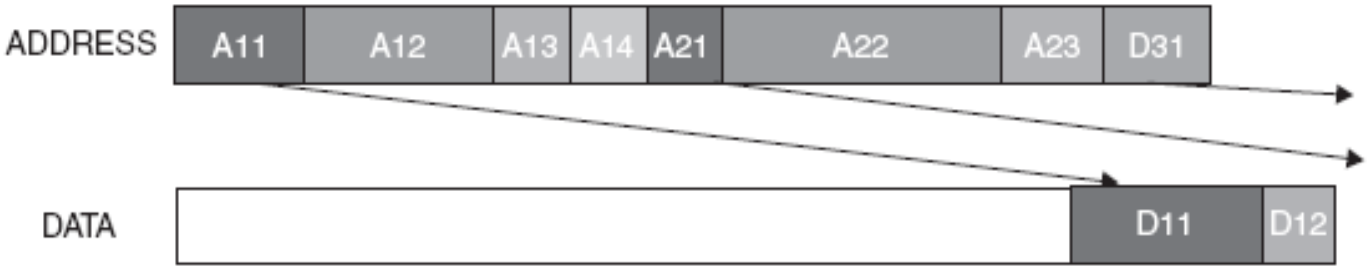
- AXI Burst
  - Simultaneous read, write transactions
  - Better bus utilization



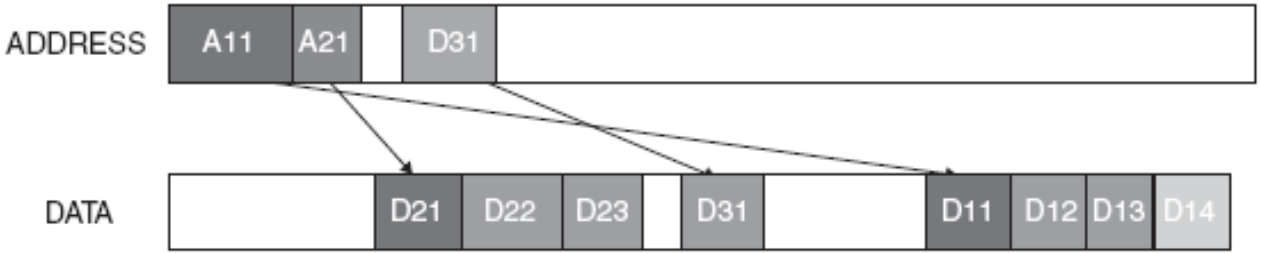
© 2008 Sudeep Pasricha & Nikil Dutt

AXI Out of Order Completion

- With AHB
  - If one slave is very slow, all data is held up
  - SPLIT transactions provide very limited improvement



- With AXI Burst
  - Multiple outstanding addresses, out of order (OO) completion allowed
  - Fast slaves may return data ahead of slow slaves



© 2008 Sudeep Pasricha & Nikil Dutt

Transaction ordering

- The AXI protocol enables out-of-order ***transaction completion***.
  - It gives an ID tag to every transaction across the interface.
  - transactions with the same ID tag are completed in order
  - transactions with different ID tags can be completed out of order.
- The interconnect can enable transactions with fast-responding slaves to complete in advance of earlier transactions with slower slaves.

Handshake process

- The **source** generates the **VALID** signal to indicate when the data or control information is available. The **destination** generates the **READY** signal to indicate that it accepts the data or control information.

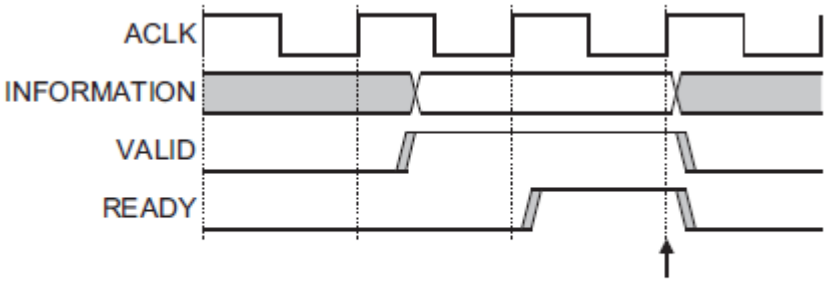


Figure 3-1 VALID before READY handshake

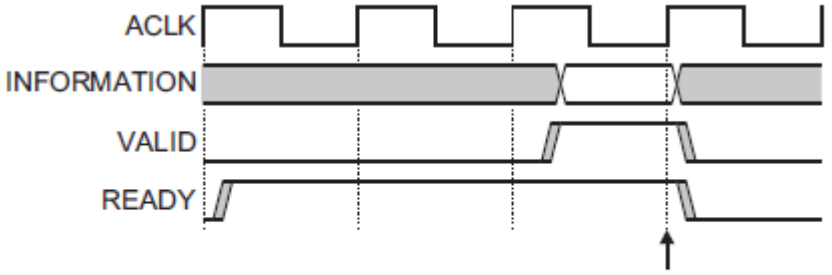


Figure 3-2 READY before VALID handshake

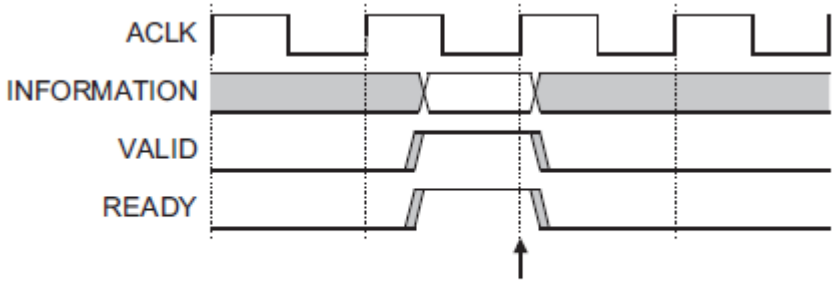
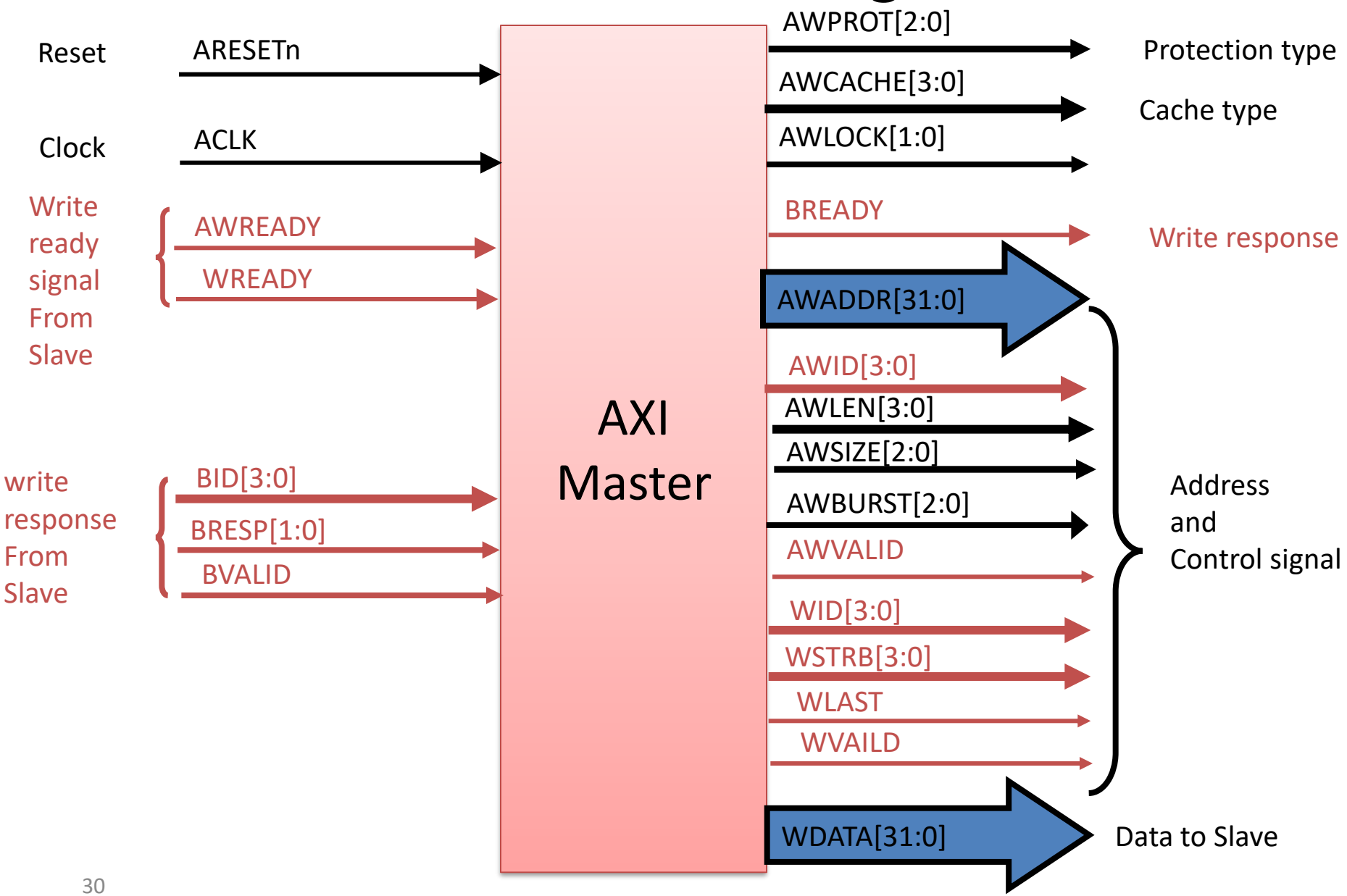


Figure 3-3 VALID with READY handshake

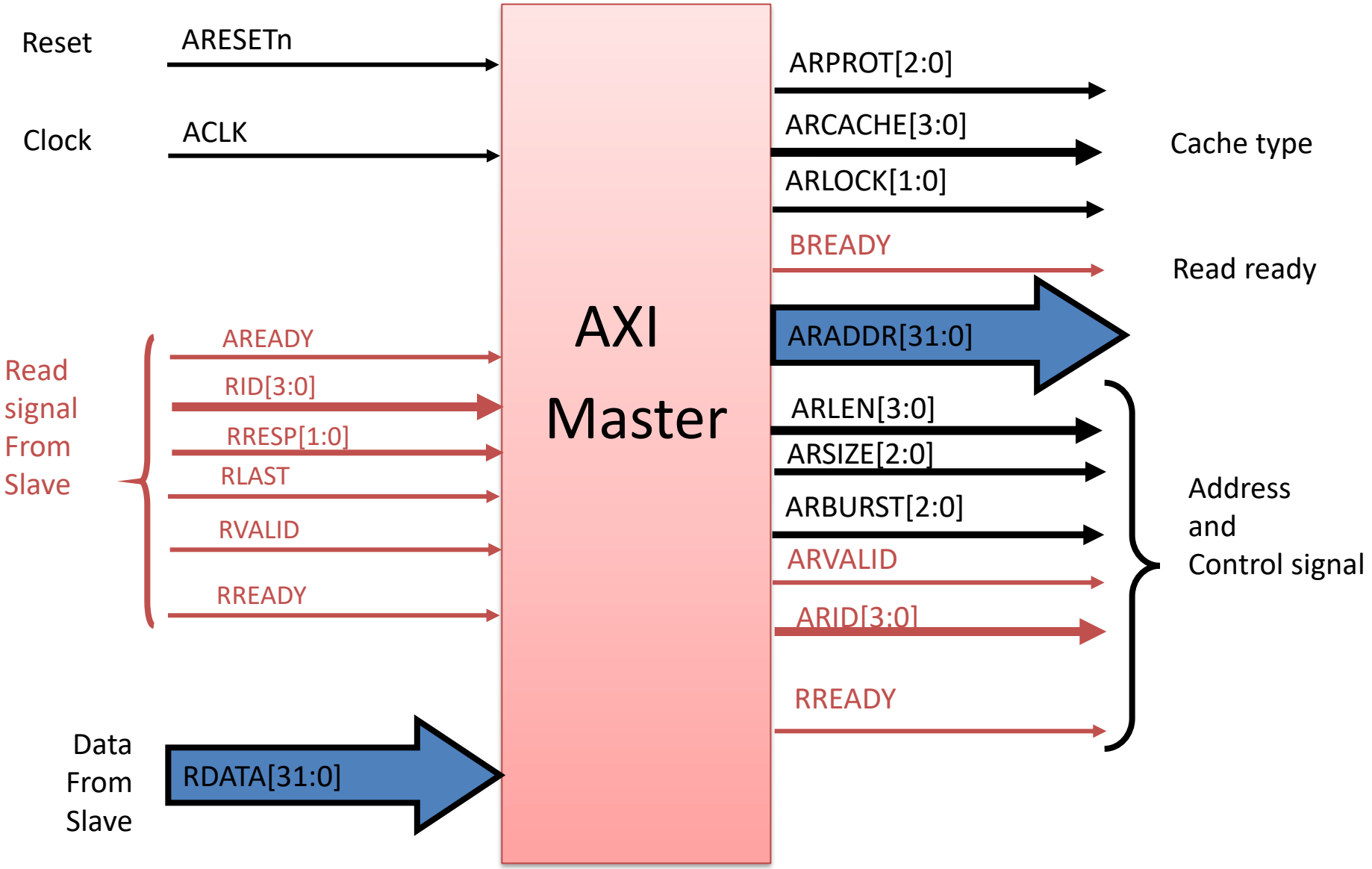
Handshake process

- The master can assert the AWVALID signal only when it drives valid address and control information. It must remain asserted until the slave accepts the address and control information and asserts the associated AWREADY signal.
  - A default AWREADY value of LOW is possible but not recommended, because it implies that the transfer takes at least two cycles, one to assert AWVALID and another to assert AWREADY.
- Relationships between the channels
  - read data must always follow the address to which the data relates
  - a write response must always follow the last write transfer in the write transaction to which the write response relates.

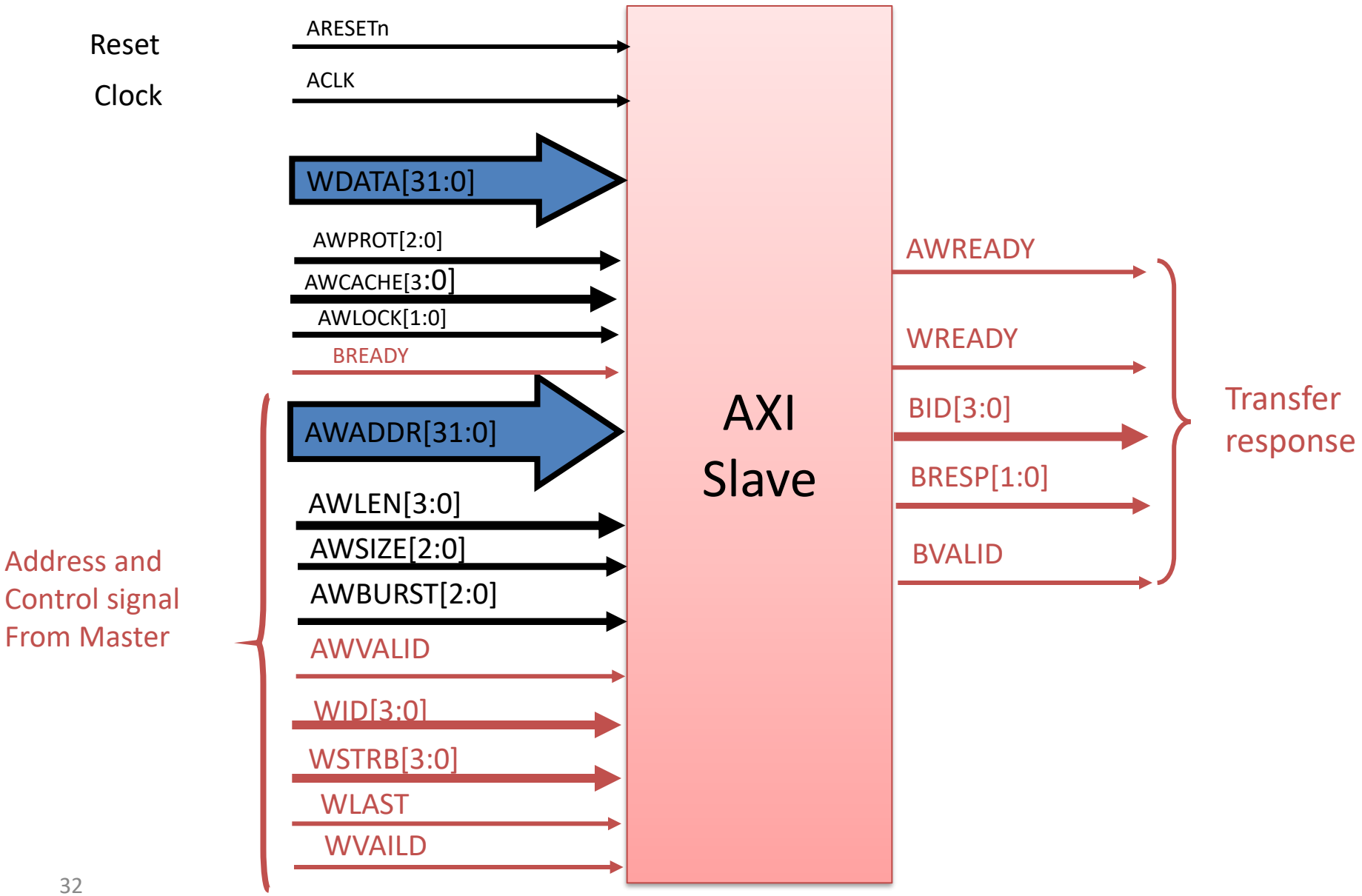
Master – WRITE signals



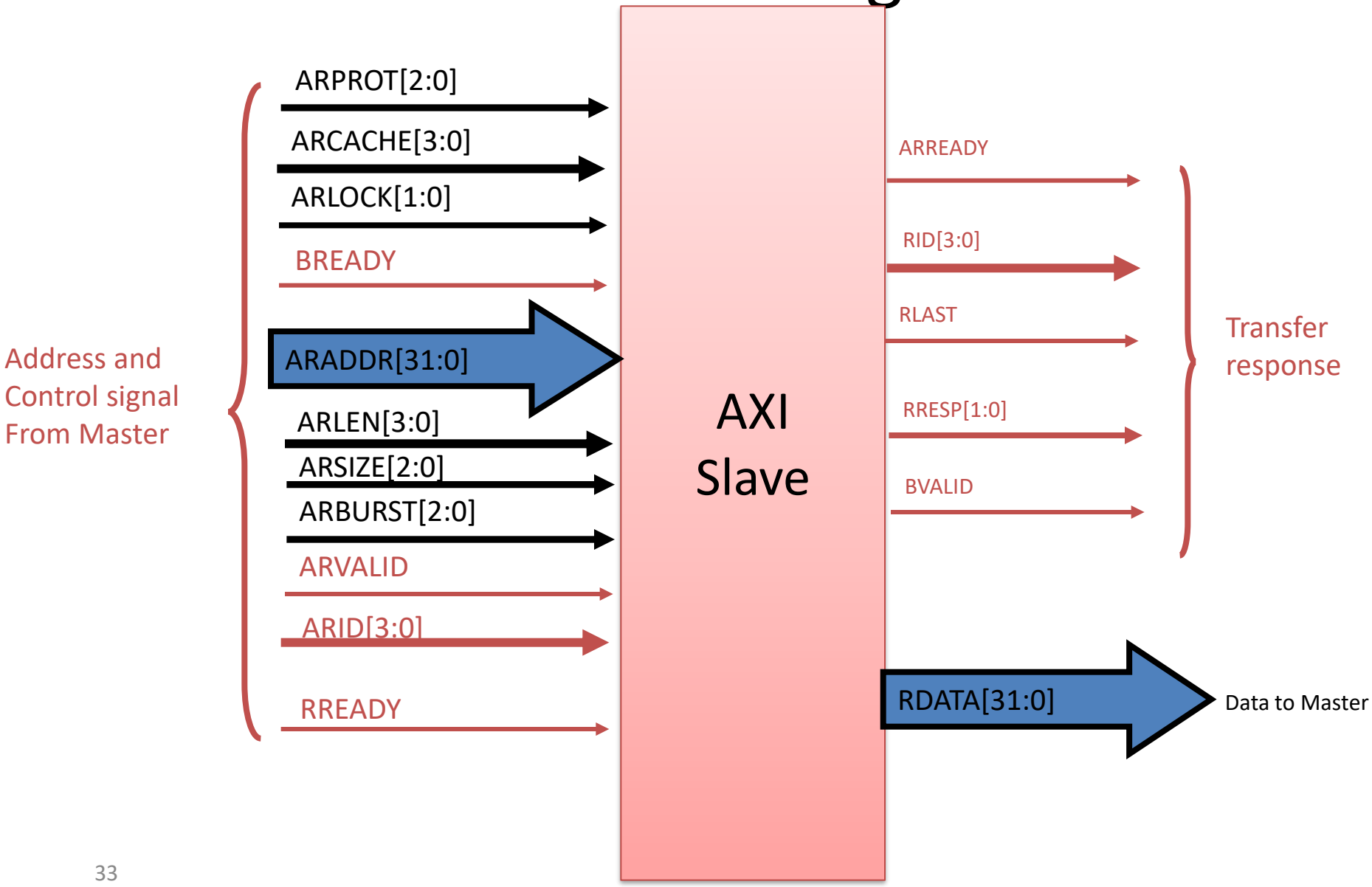
Master – READ signals



Slave – WRITE signals



Slave – READ signals



AXI signal

- global signal

Table 2-1 Global signals

Signal	Source	Description
ACLK	Clock source	Global clock signal. All signals are sampled on the rising edge of the global clock.
ARESETn	Reset source	Global reset signal. This signal is active LOW.

Write address channel signal

Signal	Source	Description
AWID[3:0]	Master	Write address ID. This signal is the identification tag for the write address group of signals.
AWADDR[31:0]	Master	Write address. The write address bus gives the address of the first transfer in a write burst transaction. The associated control signals are used to determine the addresses of the remaining transfers in the burst.
AWLEN[3:0]	Master	Burst length. The burst length gives the exact number of transfers in a burst. This information determines the number of data transfers associated with the address. See Table 4-1 on page 4-3.
AWSIZE[2:0]	Master	Burst size. This signal indicates the size of each transfer in the burst. Byte lane strobes indicate exactly which byte lanes to update. See Table 4-2 on page 4-4.
AWBURST[1:0]	Master	Burst type. The burst type, coupled with the size information, details how the address for each transfer within the burst is calculated. See Table 4-3 on page 4-5.
AWLOCK[1:0]	Master	Lock type. This signal provides additional information about the atomic characteristics of the transfer. See Table 6-1 on page 6-2.
AWCACHE[3:0]	Master	Cache type. This signal indicates the bufferable, cacheable, write-through, write-back, and allocate attributes of the transaction. See Table 5-1 on page 5-3.
AWPROT[2:0]	Master	Protection type. This signal indicates the normal, privileged, or secure protection level of the transaction and whether the transaction is a data access or an instruction access. See *Protection unit support* on page 5-5.
AWVALID	Master	Write address valid. This signal indicates that valid write address and control information are available: 1 = address and control information available 0 = address and control information not available. The address and control information remain stable until the address acknowledge signal, **AWREADY**, goes HIGH.
AWREADY	Slave	Write address ready. This signal indicates that the slave is ready to accept an address and associated control signals: 1 = slave ready 0 = slave not ready.

Write data signal

Signal	Source	Description
WID[3:0]	Master	Write ID tag. This signal is the ID tag of the write data transfer. The **WID** value must match the **AWID** value of the write transaction.
WDATA[31:0]	Master	Write data. The write data bus can be 8, 16, 32, 64, 128, 256, 512, or 1024 bits wide.
WSTRB[3:0]	Master	Write strobes. This signal indicates which byte lanes to update in memory. There is one write strobe for each eight bits of the write data bus. Therefore, **WSTRB[n]** corresponds to **WDATA[(8 × n) + 7:(8 × n)]**.
WLAST	Master	Write last. This signal indicates the last transfer in a write burst.
WVALID	Master	Write valid. This signal indicates that valid write data and strobes are available: 1 = write data and strobes available 0 = write data and strobes not available.
WREADY	Slave	Write ready. This signal indicates that the slave can accept the write data: 1 = slave ready 0 = slave not ready.

Write response signal

-

Signal	Source	Description
BID[3:0]	Slave	Response ID. The identification tag of the write response. The **BID** value must match the **AWID** value of the write transaction to which the slave is responding.
BRESP[1:0]	Slave	Write response. This signal indicates the status of the write transaction. The allowable responses are OKAY, EXOKAY, SLVERR, and DECERR.
BVALID	Slave	Write response valid. This signal indicates that a valid write response is available: 1 = write response available 0 = write response not available.
BREADY	Master	Response ready. This signal indicates that the master can accept the response information. 1 = master ready 0 = master not ready.

read address channel signal

Signal	Source	Description
ARID[3:0]	Master	Read address ID. This signal is the identification tag for the read address group of signals.
ARADDR[31:0]	Master	Read address. The read address bus gives the initial address of a read burst transaction. Only the start address of the burst is provided and the control signals that are issued alongside the address detail how the address is calculated for the remaining transfers in the burst.
ARLEN[3:0]	Master	Burst length. The burst length gives the exact number of transfers in a burst. This information determines the number of data transfers associated with the address. See Table 4-1 on page 4-3.
ARSIZE[2:0]	Master	Burst size. This signal indicates the size of each transfer in the burst. See Table 4-2 on page 4-4.
ARBURST[1:0]	Master	Burst type. The burst type, coupled with the size information, details how the address for each transfer within the burst is calculated. See Table 4-3 on page 4-5.
ARLOCK[1:0]	Master	Lock type. This signal provides additional information about the atomic characteristics of the transfer. See Table 6-1 on page 6-2.
ARCACHE[3:0]	Master	Cache type. This signal provides additional information about the cacheable characteristics of the transfer. See Table 5-1 on page 5-3.
ARPROT[2:0]	Master	Protection type. This signal provides protection unit information for the transaction. See *Protection unit support* on page 5-5.
ARVALID	Master	Read address valid. This signal indicates, when HIGH, that the read address and control information is valid and will remain stable until the address acknowledge signal, **ARREADY**, is high. 1 = address and control information valid 0 = address and control information not valid.
ARREADY	Slave	Read address ready. This signal indicates that the slave is ready to accept an address and associated control signals: 1 = slave ready 0 = slave not ready.

READ data signal

Signal	Source	Description
RID[3:0]	Slave	Read ID tag. This signal is the ID tag of the read data group of signals. The **RID** value is generated by the slave and must match the **ARID** value of the read transaction to which it is responding.
RDATA[31:0]	Slave	Read data. The read data bus can be 8, 16, 32, 64, 128, 256, 512, or 1024 bits wide.
RRESP[1:0]	Slave	Read response. This signal indicates the status of the read transfer. The allowable responses are OKAY, EXOKAY, SLVERR, and DECERR.
RLAST	Slave	Read last. This signal indicates the last transfer in a read burst.
RVALID	Slave	Read valid. This signal indicates that the required read data is available and the read transfer can complete: 1 = read data available 0 = read data not available.
RREADY	Master	Read ready. This signal indicates that the master can accept the read data and response information: 1= master ready 0 = master not ready.

Relationships between the channels

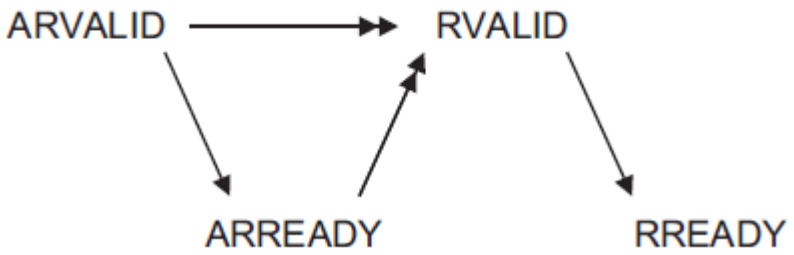
- The relationship between the address, read, write, and write response channels is flexible.
  - the write data can appear at an interface before the write address that relates to it.
  - This can occur when the write address channel contains more register stages than the write data channel.
  - the write data can appear in the same cycle as the address.
  - When the interconnect must determine the destination address space or slave space, it must realign the address and write data. This is required to assure that the write data is signaled as valid only to the slave for which it is destined.
- Two relationships that must be maintained are:
  - read data must always follow the address to which the data relates
  - a write response must always follow the last write transfer in the write transaction to which the write response relates.

Dependencies between channel handshake signals

- To prevent a deadlock situation, you must observe the dependencies that exist between the handshake signals.
  - the VALID signal of one AXI component must not be dependent on the READY signal of the other component in the transaction
  - the READY signal can wait for assertion of the VALID signal.

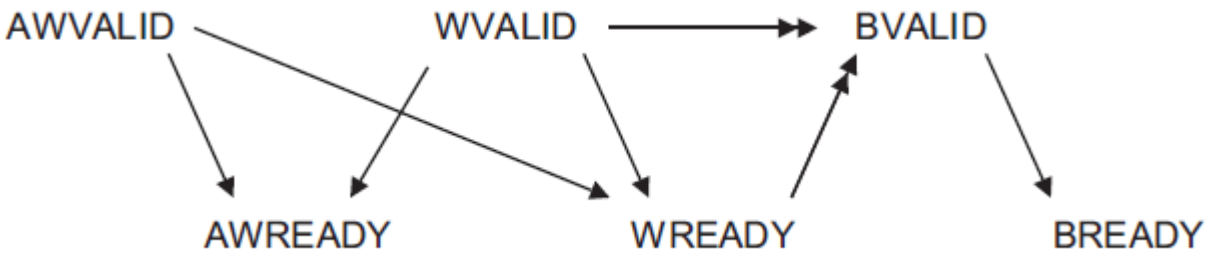
Read transaction handshake dependencies

- the slave can wait for ARVALID to be asserted before it asserts ARREADY
- the slave must wait for both ARVALID and ARREADY to be asserted before it starts to return read data by asserting RVALID.
  - The single-headed arrows point to signals that **can** be asserted before or after the previous signal is asserted
  - Double-headed arrows point to signals that **must** be asserted only after assertion of the previous signal.



Write transaction handshake dependencies

- the master must not wait for the slave to assert AWREADY or WREADY before asserting AWVALID or WVALID
- the slave can wait for AWVALID or WVALID, or both, before asserting AWREADY
- the slave can wait for AWVALID or WVALID, or both, before asserting WREADY
- the slave must wait for both WVALID and WREADY to be asserted before asserting BVALID.



Addressing options

- The AXI protocol is burst-based, and the master begins each burst by driving transfer control information and the address of the first byte in the transfer.
- As the burst transaction progresses, it is the responsibility of the slave to calculate the addresses of subsequent transfers in the burst.
- Bursts ***must not cross 4KB boundaries*** to prevent them from crossing boundaries between slaves and to limit the size of the address incrementer required within slaves.

Burst length

- Every transaction must have the number of transfers specified by ARLEN or AWLEN.
- For wrapping bursts, the length of the burst must be 2, 4, 8, or 16 transfers.

ARLEN[3:0] AWLEN[3:0]	Number of data transfers
b0000	1
b0001	2
b0010	3
. . .	
b1101	14
b1110	15
b1111	16

Burst size & type

- Fixed burst
  - Address remains the same for every transfer in the burst
  - For repeated accesses to the same location such as when loading or emptying a peripheral FIFO
- Incrementing burst
  - Address for each transfer in the burst is an increment of the previous transfer address
  - The increment value depends on the size of the transfer

ARSIZE[2:0] AWSIZE[2:0]	Bytes in transfer
b000	1
b001	2
b010	4
b011	8
b100	16
b101	32
b110	64
b111	128

ARBURST[1:0] AWBURST[1:0]	Burst type	Description	Access
b00	FIXED	Fixed-address burst	FIFO-type
b01	INCR	Incrementing-address burst	Normal sequential memory
b10	WRAP	Incrementing-address burst that wraps to a lower address at the wrap boundary	Cache line
b11	Reserved	-	-

Burst type(3/3)

- Wrapping burst
  - Similar to an incrementing burst
  - In a wrapping burst the address wraps around to a lower address when a wrap boundary is reached
  - Wrap boundary is the size of each transfer in the burst multiplied by the total number of transfers in the burst
  - Two restrictions apply to wrapping bursts:
    - the start address must be aligned to the size of the transfer
    - the length of the burst must be 2, 4, 8, or 16.

Burst address(1/4)

- We will talk about some simple formulas for determining the address and byte lanes of transfers within a burst in this section.

Start_Address	The start address issued by the master
Number_Bytes	The maximum number of bytes in each data transfer
Data_Bus_Bytes	The number of byte lanes in the data bus
Aligned_Address	The aligned version of the start address
Burst_Length	The total number of data transfers within a burst
Address_N	The address of transfer N within a burst. N is an integer from 2-16
Wrap_Boundary	The lowest address within a wrapping burst
Lower_Byte_Lane	The byte lane of the lowest addressed byte of a transfer
Upper_Byte_Lane	The byte lane of the highest addressed byte of a transfer
INT(x)	The rounded-down integer value of x

Burst address(2/4)

- Use these equations to determine addresses of transfers within a burst:
  - Start_Address = **ADDR**
  - Number_Bytes = $2^{\mathbf{SIZE}}$
  - Burst_Length = **LEN + 1**
  - Aligned_Address = (INT(Start_Address / Number_Bytes)) * Number_Bytes
- Use this equation to determine the address of the first transfer in a burst:
  - Address_1 = Start_Address
- Use this equation to determine the address of any transfer after the first transfer in a burst:
  - Address_N = Aligned_Address + (N – 1) x Number_Bytes

Burst address(3/4)

- For wrapping bursts, the Wrap_Boundary variable is used to account for the wrapping boundary:
  - Wrap_Boundary = (INT(Start_Address / (Number_Bytes * Burst_Length))) * (Number_Bytes x Burst_Length)
- If Address_N = Wrap_Boundary+(Number_Bytes * Burst_Length), use this equation:
  - Address_N = Wrap_Boundary
- After the wrapping boundary, use this equation:
  - Address_N = Start_Address + ((N – 1) * Number_Bytes) - (Number_Bytes x Burst_Length)

Burst address(4/4)

- Use these equations to determine which byte lanes to use for the first transfer in a burst:
  - Lower_Byte_Lane = Start_Address-(INT(Start_Address / Data_Bus_Bytes))*Data_Bus_Bytes
  - Upper_Byte_Lane = Aligned_Address+(Number_Bytes - 1)-(INT(Start_Address / Data_Bus_Bytes))* Data_Bus_Bytes
- Use these equations to determine which byte lanes to use for all transfers after the first transfer in a burst:
  - Lower_Byte_Lane = Address_N – (INT(Address_N / Data_Bus_Bytes)) * Data_Bus_Bytes
  - Upper_Byte_Lane=Lower_Byte_Lane+Number_Bytes – 1

Cache support

ARCACHE[3:0]
AWCACHE[3:0]

WA	RA	C	B	Transaction attributes
0	0	0	0	Noncacheable and nonbufferable
0	0	0	1	Bufferable only
0	0	1	0	Cacheable, but do not allocate
0	0	1	1	Cacheable and bufferable, but do not allocate
0	1	0	0	Reserved
0	1	0	1	Reserved
0	1	1	0	Cacheable write-through, allocate on reads only
0	1	1	1	Cacheable write-back, allocate on reads only
1	0	0	0	Reserved
1	0	0	1	Reserved
1	0	1	0	Cacheable write-through, allocate on writes only
1	0	1	1	Cacheable write-back, allocate on writes only
1	1	0	0	Reserved
1	1	0	1	Reserved
1	1	1	0	Cacheable write-through, allocate on both reads and writes
1	1	1	1	Cacheable write-back, allocate on both reads and writes

Protection unit

Table 5-2 Protection encoding

ARPROT[2:0] AWPROT[2:0]	Protection level
[0]	1 = privileged access 0 = normal access
[1]	1 = nonsecure access 0 = secure access
[2]	1 = instruction access 0 = data access

Atomic accesses - signal

- Source: Master
  - Atomic access encoding

ARLOCK[1:0] AWLOCK[1:0]	Access type
b00	Normal access
b01	Exclusive access
b10	Locked access
b11	Reserved

- Source: slave
  - **RRESP[1:0]** or **BRESP[1:0]** signal indicates the success or failure of the exclusive access.
    - EXOKAY
    - OKAY

Atomic Exclusive access

1. A master performs an exclusive read from an address location.
2. At some later time, the master attempts to complete the exclusive operation by performing an exclusive write to the same address location.
3. The exclusive write access of the master is signalled as
   - Successful if no other master has written to that location between the read and write accesses.
   - Failed if another master has written to that location between the read and write accesses

- A master receives the EXOKAY response from the slave, indicating that the slave recorded the address to be monitored.

Atomic access

- Locked access
  - Only the master is allowed access to the slave until an locked transfer from the same master complete
  - Disadvantage
    - Impact on the interconnect performance
- Exclusive access
  - without requiring the bus to remain locked to a particular master for the duration of the operation.
  - Advantage
    - do not impact either the critical bus access latency or the maximum achievable bandwidth.

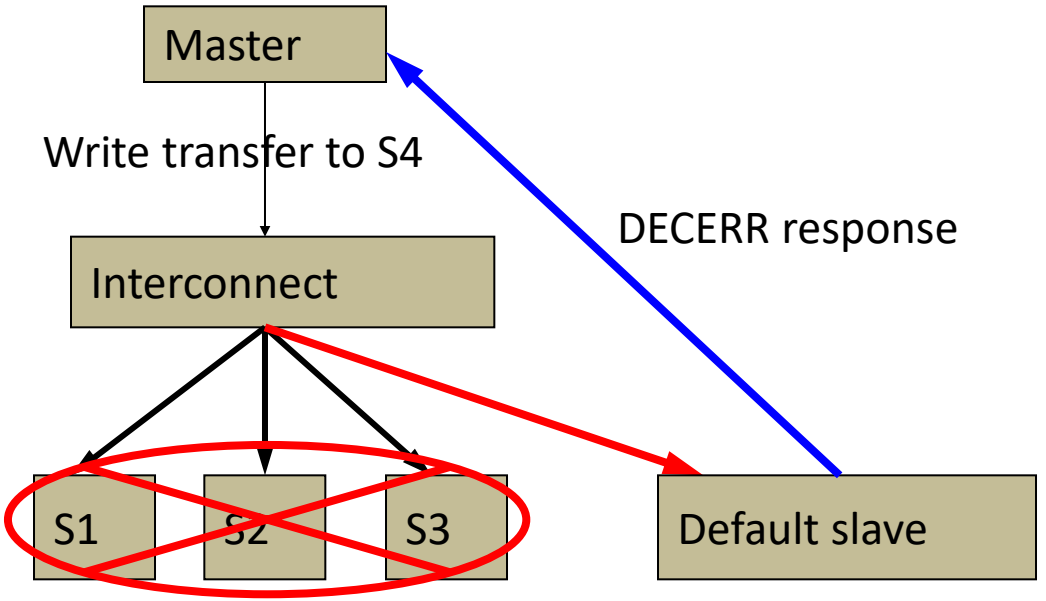
Response Signals

RRESP[1:0] BRESP[1:0]	Response	Meaning
b00	OKAY	Normal access okay indicates if a normal access has been successful. Can also indicate an exclusive access failure.
b01	EXOKAY	Exclusive access okay indicates that either the read or write portion of an exclusive access has been successful.
b10	SLVERR	Slave error is used when the access has reached the slave successfully, but the slave wishes to return an error condition to the originating master.
b11	DECERR	Decode error is generated typically by an interconnect component to indicate that there is no slave at the transaction address.

- If the master attempts an exclusive read from a slave that does not support exclusive accesses, the slave returns the OKAY response instead of the EXOKAY response.
- The master can treat this as an error condition indicating that the exclusive access is not supported.

Response Signal (cont.)

- The required number of data must be performed, even if an error is reported.
- Default slave
  - When the interconnect cannot successfully decode a slave access , It routes the access to default slave, and default slave return DECERR response



Difference between read and write response signal

- Write transfer
  - Just one response given for the entire burst.
  - Response signal transfer by write response channel
- Read transfer
  - Can give different responses for different transfer within a burst
  - Response signal transfer by read data channel

Normal access success

The OKAY response indicates:

- the success of a normal access
- the failure of an exclusive access
- an exclusive access to a slave that does not support exclusive access.

OKAY is the response for most transactions

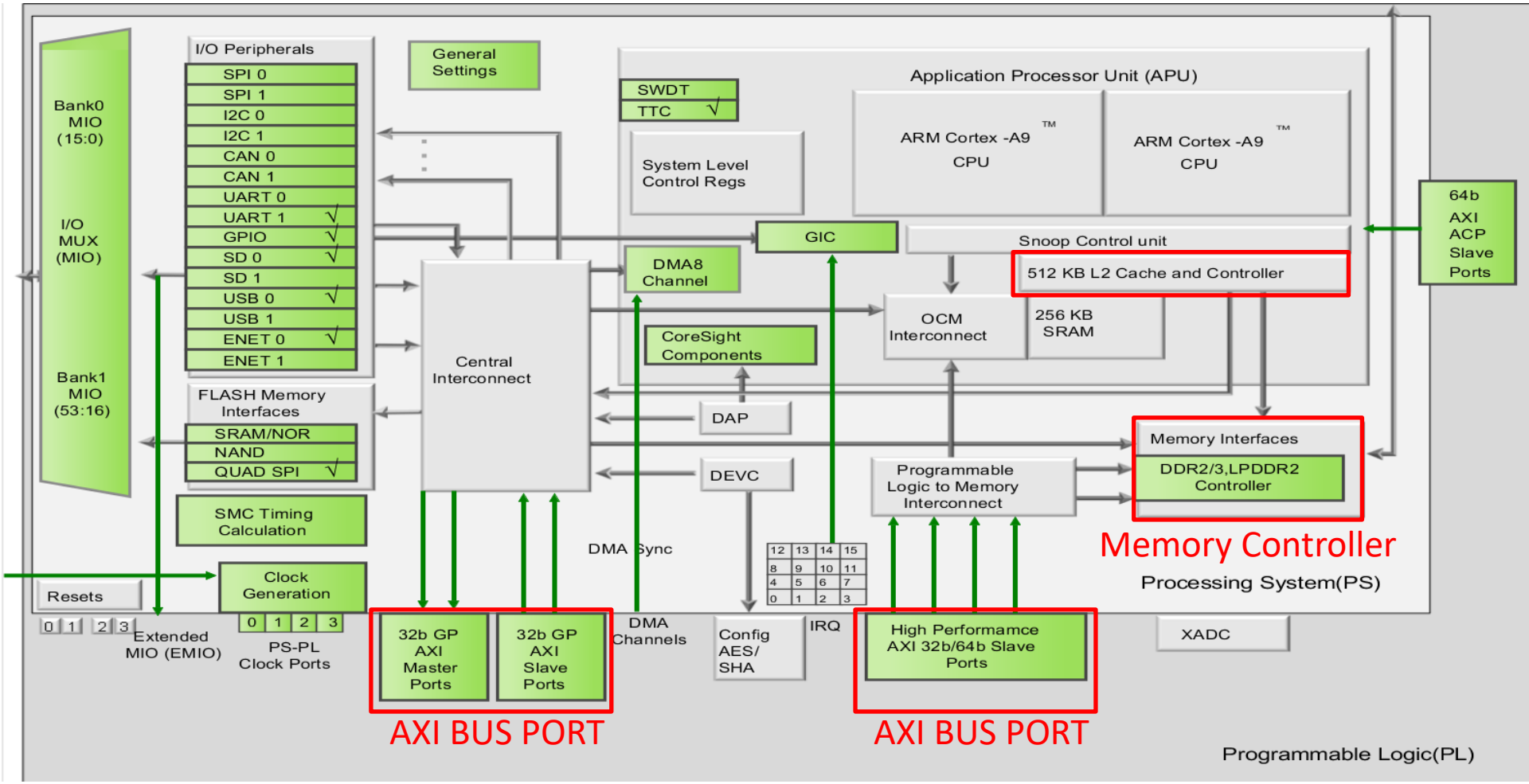
Slave error

The SLVERR response indicates an unsuccessful transaction. Examples of slave error conditions are:

- FIFO/buffer overrun or underrun condition
- unsupported transfer size attempted
- write access attempted to read-only location
- timeout condition in the slave
- access attempted to an address where no registers are present
- access attempted to a disabled or powered-down function

Zynq Bus Wrapper

ZYNQ-7000


Bank0 MIO (15:0)
I/O MUX (MIO)
Bank1 MIO (53:16)
I/O Peripherals
SPI 0
SPI 1
I2C 0
I2C 1
CAN 0
CAN 1
UART 0
UART 1 √
GPIO √
SD 0 √
SD 1
USB 0 √
USB 1
ENET 0 √
ENET 1
FLASH Memory Interfaces
SRAM/NOR
NAND
QUAD SPI √
SMC Timing Calculation
General Settings
SWDT
TTC √
System Level Control Regs
Application Processor Unit (APU)
ARM Cortex -A9 CPU
ARM Cortex -A9 CPU
GIC
Snoop Control unit
512 KB L2 Cache and Controller
64b AXI ACP Slave Ports
DMA8 Channel
OCM Interconnect
256 KB SRAM
Central Interconnect
CoreSight Components
DAP
DEVC
Memory Interfaces
DDR2/3,LPDDR2 Controller
Programmable Logic to Memory Interconnect
Memory Controller
Processing System(PS)
DMA Sync
Resets
Clock Generation
0 1 2 3
Extended MIO (EMIO)
PS-PL Clock Ports
32b GP AXI Master Ports
32b GP AXI Slave Ports
DMA Channels
Config AES/ SHA
IRQ
High Performamce AXI 32b/64b Slave Ports
XADC
AXI BUS PORT
AXI BUS PORT
Programmable Logic(PL)

PS-PL interfaces

- AXI_HP
  - four high performance/bandwidth master ports on the PS AXI interconnect
- AXI_GP, four general purpose ports
  - Two, 32-bit master interfaces
  - Two, 32-bit slave interfaces
  - Asynchronous clock frequency domain crossing for all AXI interfaces between the PL and PS
  - Standard AXI 3.0 interfaces supported
- AXI_ACP
  - One 64-bit cache coherent slave port in the APU interfaces to a PL master port
  - Connects to the snoop control unit for cache coherency between the CPUs and the PL

Table 22-8: **Data Movement Method Comparison Summary**

Method	Benefits	Drawbacks	Suggested Uses	Estimated Throughput
CPU Programmed I/O	• Simple Software • Least PL Resources • Simple PL Slaves	• Lowest Throughput	• Control Functions	<25 MB/s
PS DMAC	• Least PL Resources • Medium Throughput • Multiple Channels • Simple PL Slaves	• Somewhat complex DMA programming	• Limited PL Resource DMAs	600 MB/s
PL AXI_HP DMA	• Highest Throughput • Multiple Interfaces • Command/Data FIFOs	• OCM/DDR access only • More complex PL Master design	• High Performance DMA for large datasets	1,200 MB/s (per interface)
PL AXI_ACP DMA	• Highest Throughput • Lowest Latency • Optional Cache Coherency	• Large burst might cause cache thrashing • Shares CPU Interconnect bandwidth • More complex PL Master design	• High Performance DMA for smaller, coherent datasets • Medium granularity CPU offload	1,200 MB/s
PL AXI_GP DMA	• Medium Throughput	• More complex PL Master design	• PL to PS Control Functions • PS I/O Peripheral Access	600 MB/s

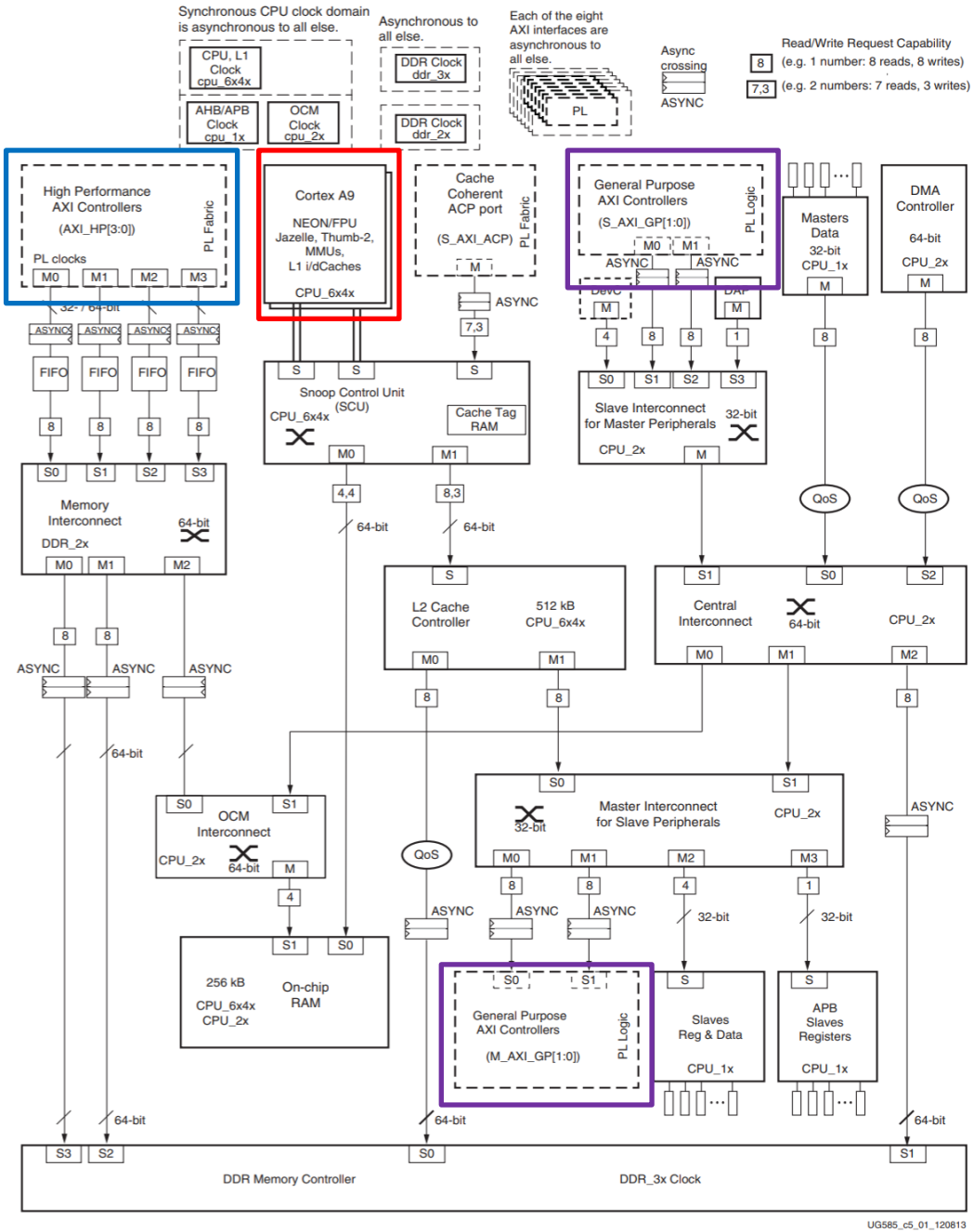


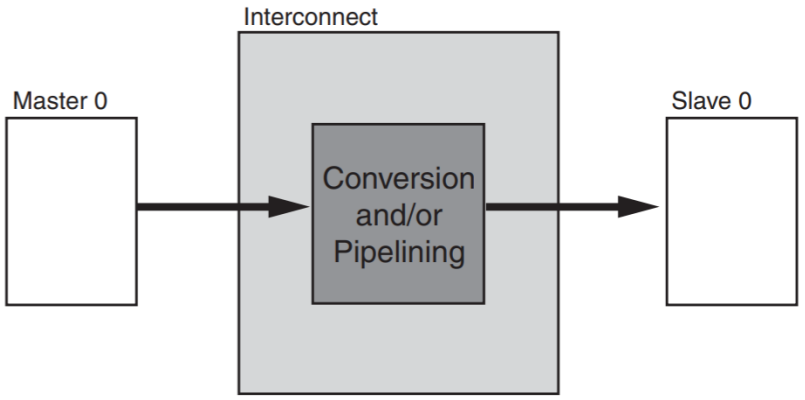
Figure 5-1: **Interconnect Block Diagram**

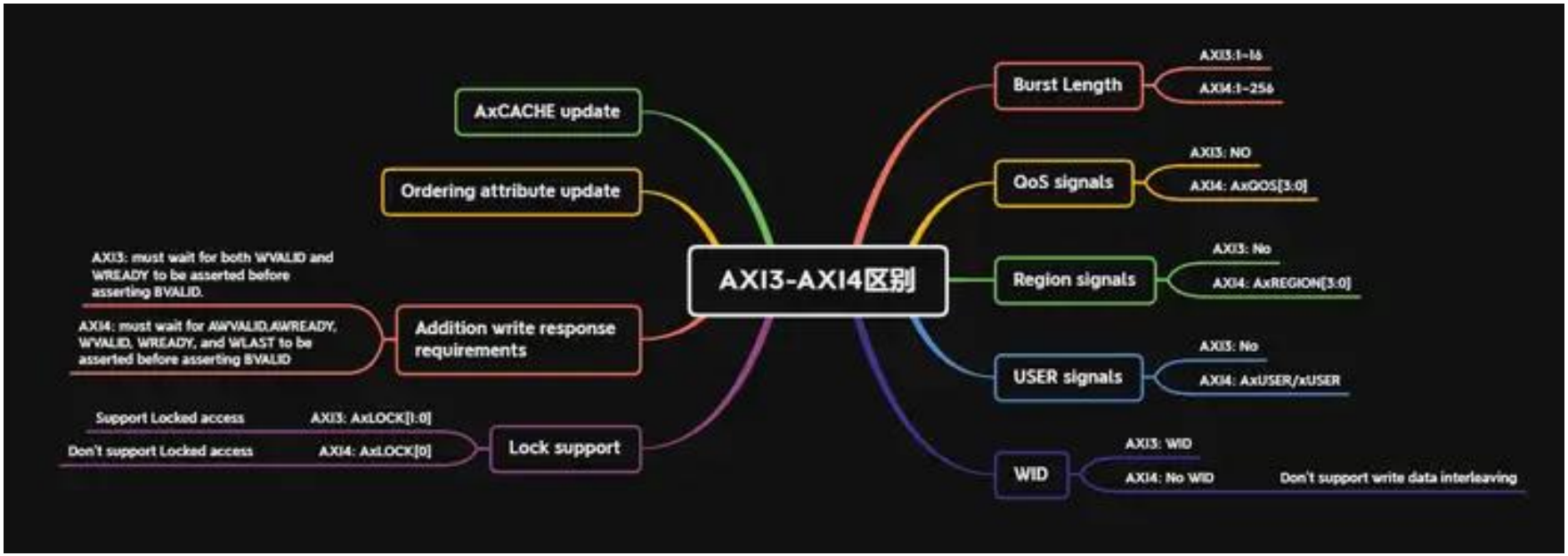
AXI Interconnect Core Use Models

- The AXI Interconnect IP core connects one or more AXI memory-mapped master devices to one or more memory-mapped slave devices.
- Possible use cases:
  - Conversion Only
  - N-to-1 Interconnect
  - 1-to-N Interconnect
  - N-to-M Interconnect (Sparse Crossbar Mode)

Conversion Only

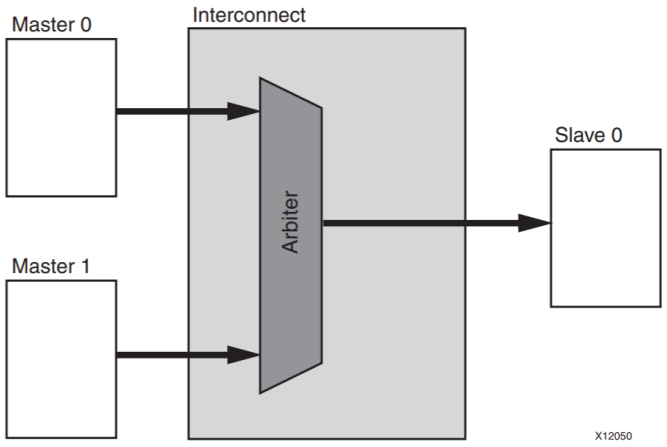
- The AXI Interconnect core can perform various conversion and pipelining functions when connecting one master device to one slave device. These are:
  - Data width conversion
  - Clock rate conversion
  - AXI4-Lite slave adaptation
  - AXI-3 slave adaptation
  - Pipelining, such as a register slice or data channel FIFO



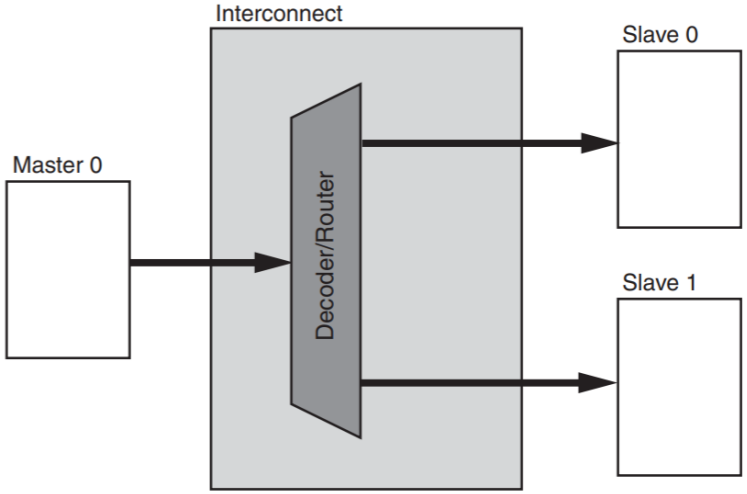

AXI3-AXI4区别
AxCACHE update
Ordering attribute update
Addition write response requirements
AXI3: must wait for both WVALID and WREADY to be asserted before asserting BVALID.
AXI4: must wait for AWVALID,AWREADY, WVALID, WREADY, and WLAST to be asserted before asserting BVALID
Lock support
Support Locked access
AXI3: AxLOCK[1:0]
Don't support Locked access
AXI4: AxLOCK[0]
Burst Length
AXI3:1~16
AXI4:1~256
QoS signals
AXI3: NO
AXI4: AxQOS[3:0]
Region signals
AXI3: No
AXI4: AxREGION[3:0]
USER signals
AXI3: No
AXI4: AxUSER/xUSER
WID
AXI3: WID
AXI4: No WID
Don't support write data interleaving

Other scenarios

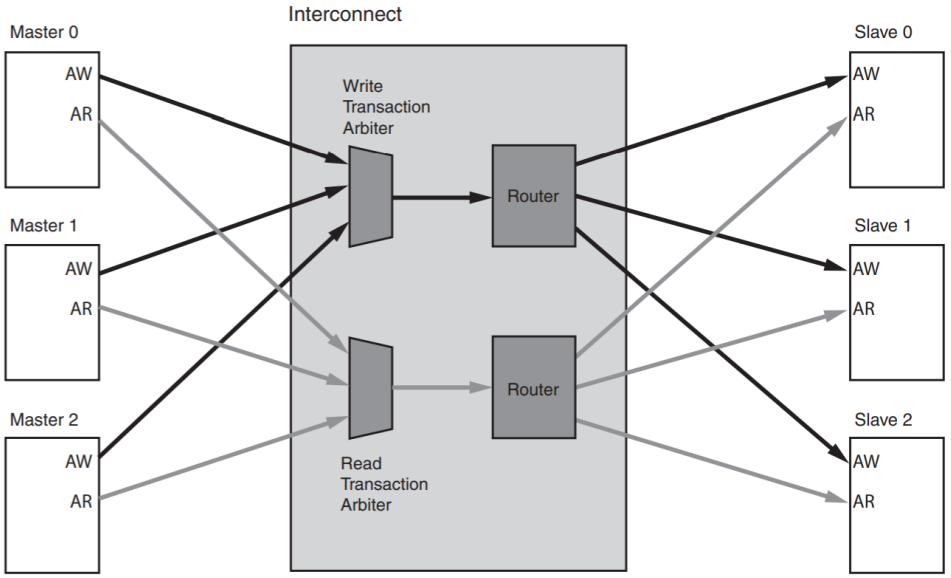
- N-to-1 AXI Interconnect



- 1-to-N AXI Interconnect



- Shared Write and Read Address Arbitrations



PS-PL interfaces

- AXI_HP, four high performance/bandwidth master ports on the PS AXI interconnect
- 32-bit or 64-bit data master interfaces (independently programmed)
- Efficient resizing in 32-bit slave interface configuration mode
- Efficient upsizing to 64-bits for aligned 32-bit transfers in 32-bit slave interface configuration mode
- Automatic expansion to 64 bits for unaligned 32-bit transfers in 32-bit slave interface configuration mode
- Dynamic command upsizing translation between 32-bit and 64-bit interfaces, controllable through AxCACHE[1]
- Separate R/W programmable issuing capability for read and write commands
- Programmable release threshold of write commands
- Asynchronous clock frequency domain crossing for all AXI interfaces between the PL and PS
- Smoothing out of “long-latency” transfers using 1 KB (128 by 64 bits) data FIFOs for both reads and writes
- QoS signaling available from PL ports
- Command and data FIFO fill-level counts available to the PL
- Standard AXI 3.0 interfaces supported
- Large slave interface read acceptance capability in the range of 14 to 70 commands (burst length dependent)
- Large slave interface write acceptance capability in the range of 8 to 32 commands (burst length dependent)

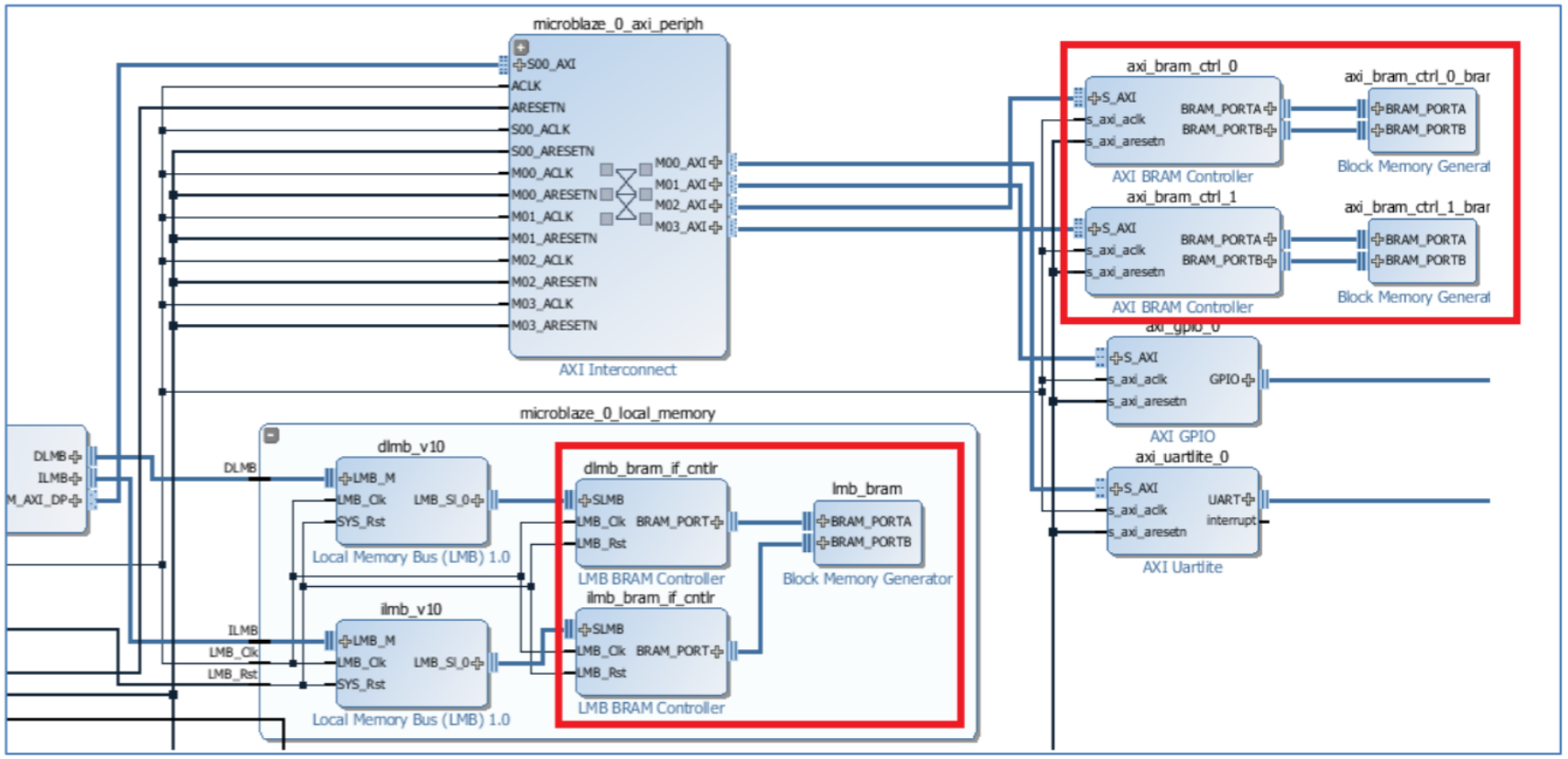
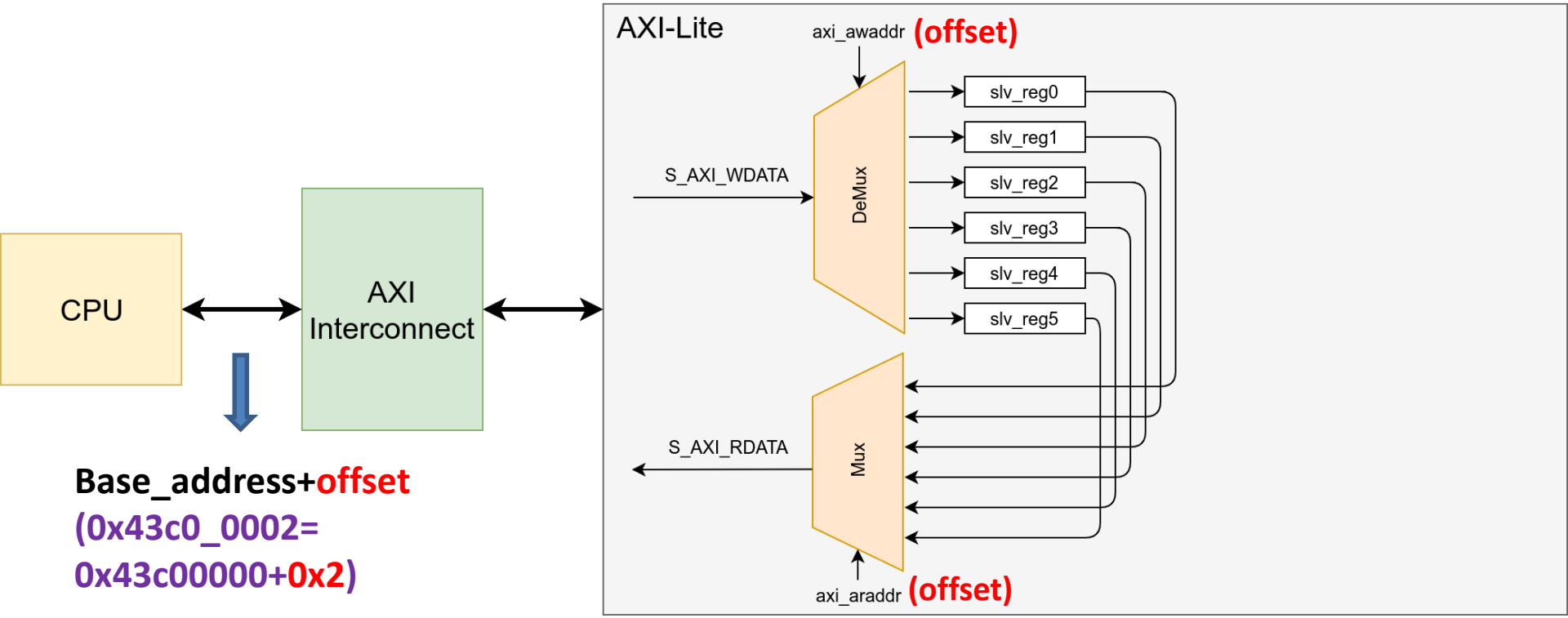


Diagram | Address Editor

Cell	Slave Interface	Base Name	Offset Address	Range	High Address
microblaze_0					
Data (32 address bits : 4G)					
microblaze_0_local_memory/dlmb_bram_if_cntlr	SLMB	Mem	0x00000000	8K	0x00001FFF
axi_uartlite_0	S_AXI	Reg	0x40600000	64K	0x4060FFFF
axi_gpio_0	S_AXI	Reg	0x40000000	64K	0x4000FFFF
axi_bram_ctrl_0	S_AXI	Mem0	0xC0000000	8K	0xC0001FFF
axi_bram_ctrl_1	S_AXI	Mem0	0xC2000000	8K	0xC2001FFF
Instruction (32 address bits : 4G)					
microblaze_0_local_memory/ilmb_bram_if_cntlr	SLMB	Mem	0x00000000	8K	0x00001FFF

ZYNQ <-> Custom IP

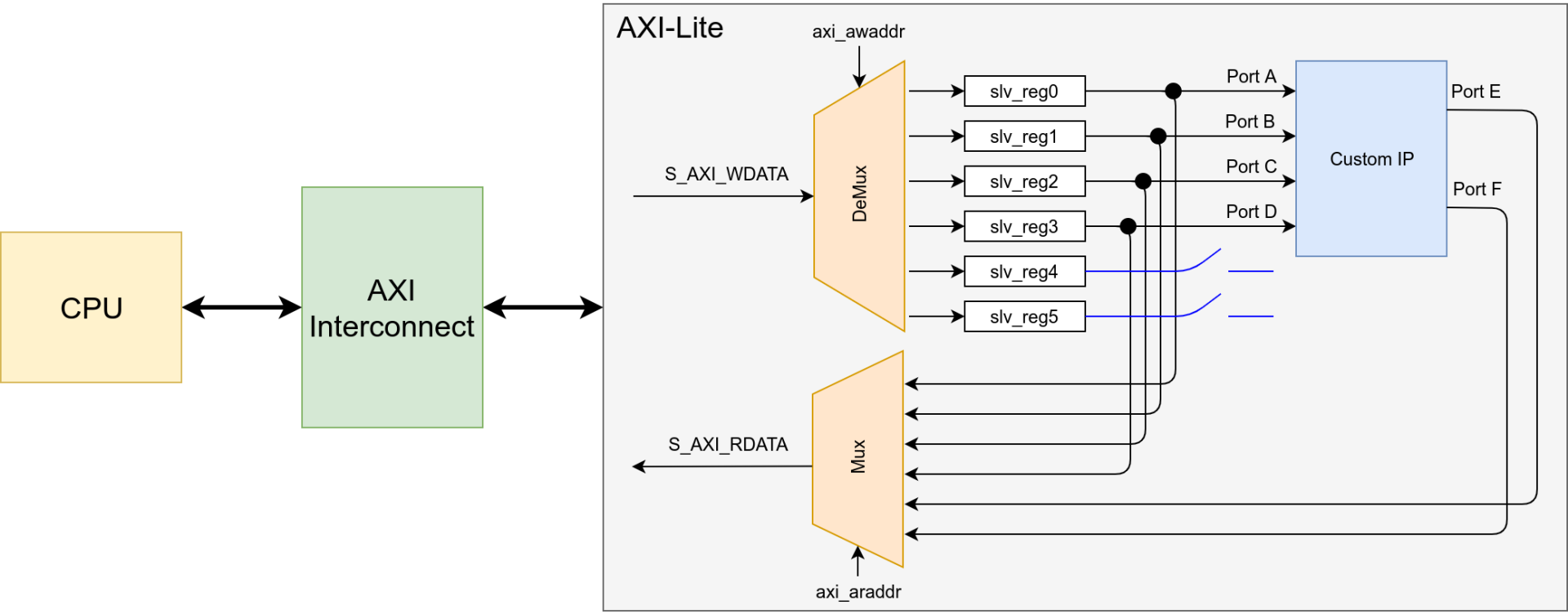
Zynq 的電路裡面含有CPU、Memory等，CPU若要跟客製化電路進行溝通需要使用特定協議，這邊由於Zynq是使用ARM的處理器，所以這邊所使用的BUS協定為AXI BUS的協定。



Xilnx Vivado生成的AXI-Lite模板

ZYNQ <-> Custom IP

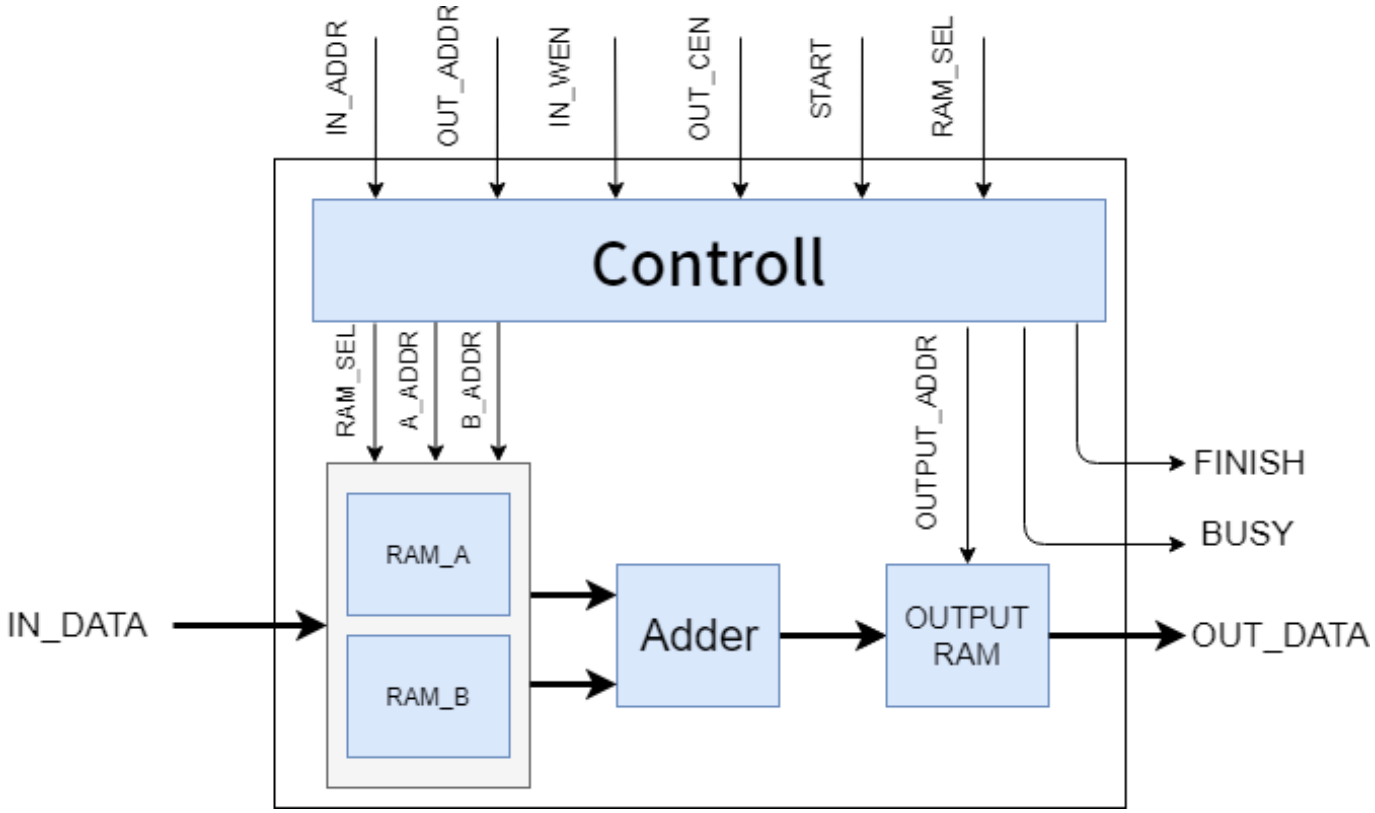
Xilinx Vivado可以生成依循AXI協定的控制電路，可以修改此控制電路，使ZYNQ與客製化電路彼此可以進行資料傳輸與控制。



使客製化硬體依循AXI-Lite協定

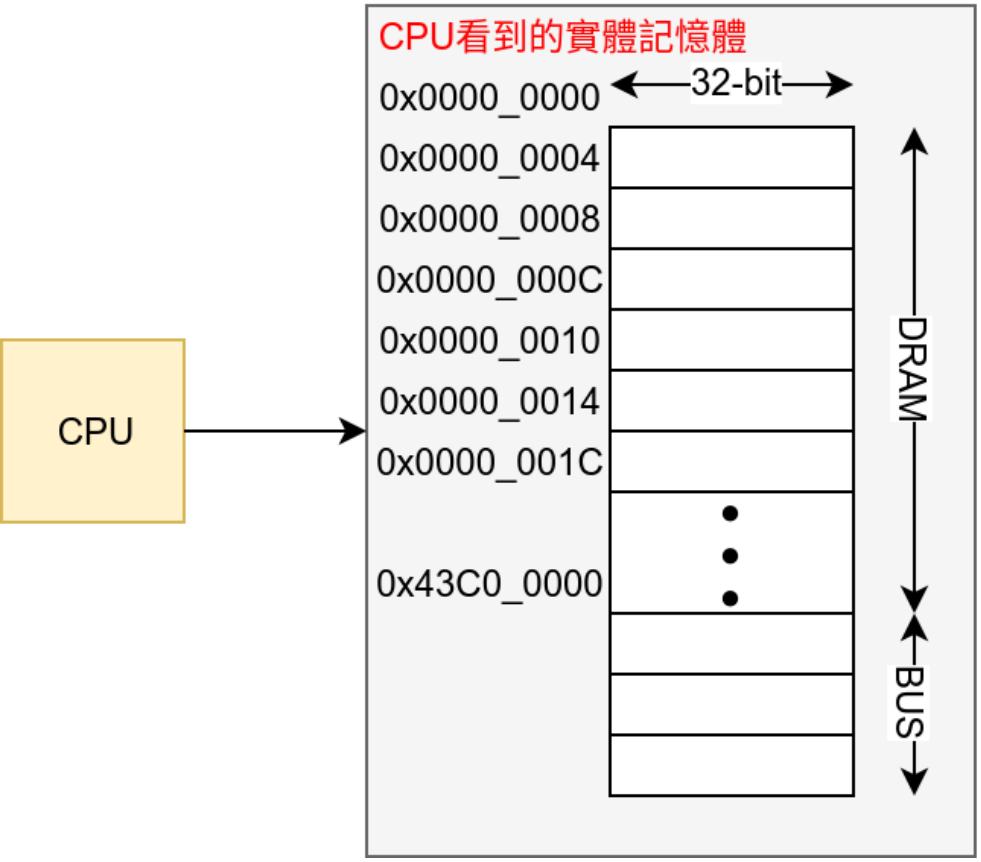
Adder IP

加法電路，將RAM_A以及RAM_B同一個記憶體位址的值相加，在儲存至相對應的OUTPUT RAM記憶體位址中。

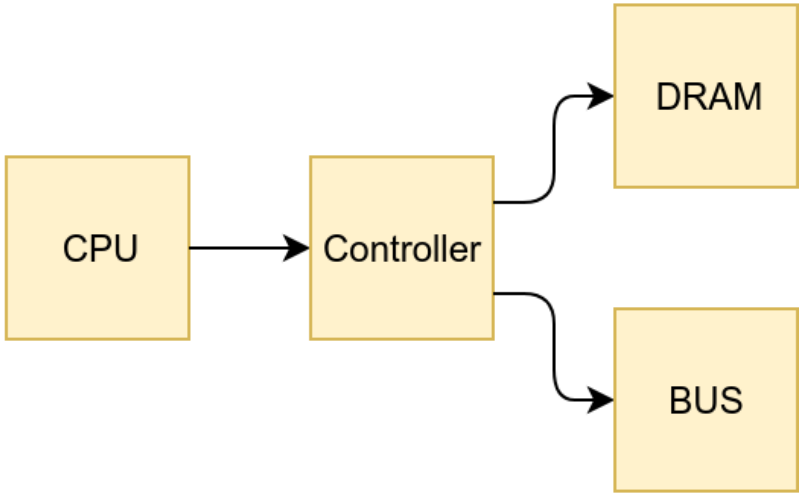


如何讓CPU與客製化硬體進行溝通

對CPU來說它只需要將值從Cache寫到某個實體記憶體位置，不需要知道他是寫到DRAM還是BUS，寫到DRAM還是BUS是透過內部控制器來決定的。

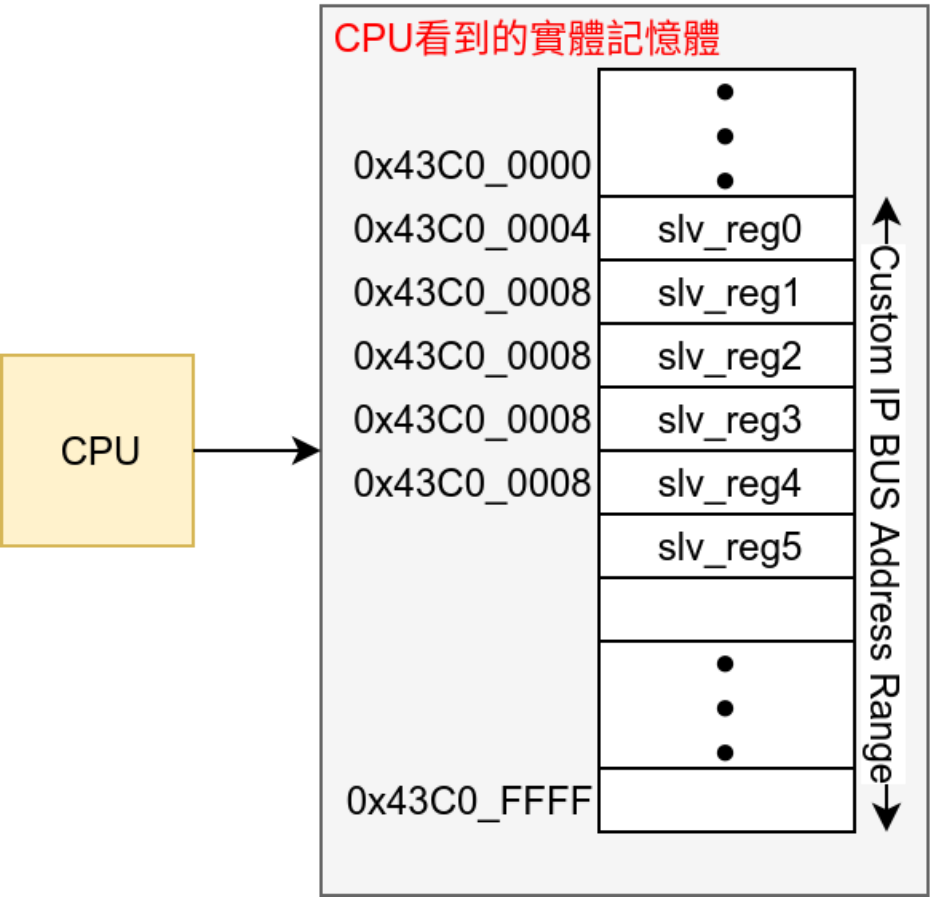


實際上的運行方式



使客製化硬體依循AXI-Lite協定

CPU與客製化電路之間的傳輸



實際上的運行方式



How to access the peripherals

```
main() {
   volatile *ptr;
   ……
   ptr = 0x20000000;
   *ptr = 18;
   ……
}
```

```
   ……
   MOV r1, #18;
   LDR r2,=0x20000000
   STR r1, [r2]
   ……
```

